

교과목 3 : 초거대언어모델(LLM)

단원 4 : 프롬프트 엔지니어링

DAY6. LangGraph 기반 멀티 에이전트 구조와 응용

목 차

1. Agent vs Workflow		3
2. LangGraph API		27
3. Multi agent concept		58

1. Agent vs Workflow

자유로운 게 항상 좋은 건 아니다

지금까지는 LLM 이 무엇을 언제 할지 스스로 결정하는 자유로운 에이전트였습니다. 탐색적 작업엔 강력하지만, "반드시 분류 후 배정" 같은 정해진 업무에는 오히려 고정 워크플로우가 맞습니다. 예측 가능하고 검증·감사가 쉽거든요.

1. 자유 에이전트의 그림자

`create_agent` 는 LLM 이 자유롭게 도구를 골라 일했습니다. 강력하지만, 업무 자동화에는 단점이 있습니다.

자유 에이전트의 단점:

- 예측 불가: 매 실행마다 도구 호출 순서 · 횟수가 달라질 수 있음
- 검증 어려움: "반드시 분류 후 배정"이라는 규칙을 강제하기 어려움
- 비용 변동: LLM이 루프를 몇 번 돌지 모름

탐색적 질문("이 고객에게 뭘 추천할까?")엔 이 자유로움이 강점입니다. 하지만 정형화된 반복 업무엔 오히려 독이 될 수 있죠.

2. 정해진 순서가 중요한 일

CS 티켓 처리를 생각해 봅시다.

티켓 처리 업무:

1. 분류 (무슨 문의인가?)
 2. 우선순위 판단 (얼마나 급한가?)
 3. 담당팀 배정 (누가 처리하나?)
- 이 순서는 반드시 지켜져야 함!

이건 순서가 정해진 업무입니다. "분류 안 하고 배정"하거나 "우선순위 건너뛰기"는 안 됩니다. 회사 규칙이니까요. 그런데 자유 에이전트는 이 순서를 보장하지 못합니다 — LLM이 가끔 순서를 건너뛰거나 바꿀 수 있죠.

3. 워크플로우 — 정해진 파이프라인

이런 업무엔 워크플로우(고정된 파이프라인)가 맞습니다.

자유 에이전트: LLM이 "다음에 뭐 할까?" 매번 결정 (자유, 예측불가)
워크플로우: 개발자가 "분류 → 우선순위 → 배정" 순서를 고정 (정해짐, 예측가능)

워크플로우는 우리가 흐름을 미리 정합니다. "분류 다음엔 우선순위, 그 다음엔 배정"이라고 코드로 고정하죠. 그러면 항상 같은 순서로 처리되고, 검증·감사가 쉽습니다.

비유: 자유 에이전트는 재량껏 일하는 베테랑(상황 따라 융통성), 워크플로우는 체크리스트대로 일하는 신입(정해진 절차)입니다. 응급 상황엔 베테랑이 낫지만, "서류는 반드시 이 순서로 처리"하는 정형 업무엔 체크리스트가 안전하죠. 빠뜨림이 없으니까요.

4. 백엔드의 상태 머신

백엔드 개발자에게 워크플로우는 익숙한 개념입니다 — 상태 머신/처리 파이프라인입니다.

입력 → [단계 1] → [단계 2] → [단계 3] → 출력
각 단계가 상태를 받아 일부를 채우고 다음으로 넘김

주문 처리(접수 → 결제 → 배송 → 완료), 회원가입(입력 → 검증 → 저장 → 메일) 같은 게 다 이런 파이프라인이죠. LangGraph의 `StateGraph`로 이걸 만듭니다.

5. 이 강에서 만들 것

CS 티켓을 분류 → 우선순위 → 배정 순서로 처리하는 워크플로우를 만듭니다.

티켓 "결제가 안 돼요"
→ 분류: 결제 (LLM)
→ 우선순위: 긴급 (규칙)
→ 배정: 결제지원팀 (규칙)

핵심 정리

- 자유 에이전트는 **예측 불가·검증 어려움·비용 변동**의 단점.
- **정해진 순서**가 중요한 업무엔 **고정 워크플로우**가 맞다.
- 워크플로우 = 흐름을 미리 정한 **상태 머신/파이프라인**.
- **일부 노드만 LLM**, 나머지는 규칙(분업).

에이전트 vs 워크플로우

ReAct 에이전트는 LLM 이 흐름을 동적으로 결정(자유·예측 불가), **StateGraph 워크플로우**는 개발자가 흐름을 엣지로 고정(정해짐·예측 가능)합니다. 탐색에는 에이전트, 정형 업무에는 워크플로우가 어울립니다.

1. 두 방식 비교

구분	ReAct 에이전트	StateGraph 워크플로우
흐름 제어	LLM이 동적으로 결정	개발자가 엣지로 고정
예측성	낮음(실행마다 다름)	높음(항상 같은 경로)
적합 작업	탐색·개방형 질문	정형화된 반복 업무
디버깅	어려움	노드 단위로 쉬움
LLM 사용	거의 모든 단계	필요한 노드에서만

2. 흐름을 누가 정하나

가장 큰 차이는 "흐름(순서)을 누가 정하느냐"입니다.

ReAct 에이전트: LLM이 정함

질문 → LLM "검색하자" → LLM "또 검색하자" → LLM "이제 답하자"
(매번 LLM이 다음 행동 결정 → 실행마다 달라질 수 있음)

StateGraph 워크플로우: 개발자가 정함

입력 → [분류] → [우선순위] → [배정] → 출력
(엣지로 순서 고정 → 항상 이 경로)

에이전트는 LLM이 매 순간 "다음에 뭐 할까?"를 정하니 자유롭지만 예측 불가입니다. 워크플로우는 우리가 엣지로 "이 다음엔 이거"라고 못 박으니 항상 같은 경로죠.

3. 예측성의 가치

"예측 가능"이 왜 중요할까요? 업무 자동화에서는 결정적입니다.

예측 불가(에이전트): 같은 티켓도 실행마다 다르게 처리될 수 있음

- "왜 이 티켓은 배정 안 됐지?" 디버깅 어려움
- 감사(audit): "이 처리가 규칙대로 됐나?" 확인 어려움

예측 가능(워크플로우): 같은 입력 → 항상 같은 경로 · 결과

- 디버깅 쉬움 (어느 노드에서 문제인지 명확)
- 감사 쉬움 (규칙대로 처리됨이 보장)

금융·의료·법무처럼 "규칙대로 처리됐음을 증명" 해야 하는 분야에서는 예측 가능한 워크플로우가 필수입니다. "AI가 알아서 했어요"는 통하지 않죠.

4. 디버깅 — 노드 단위

워크플로우는 노드 단위로 나뉘어 디버깅이 쉽습니다.

에이전트: 한 덩어리 → "어디서 잘못됐지?" 찾기 어려움

워크플로우: [분류] [우선순위] [배정] 각 단계가 분리

→ "분류는 맞는데 우선순위가 틀렸네" → `priority_node`만 고치면 됨

각 단계가 작은 함수(노드)라, 어느 단계에서 문제인지 즉시 보입니다.

단계별 로깅·예외처리·재시도도 깔끔히 붙일 수 있죠.

5. LLM은 필요한 곳에만

워크플로우의 또 다른 장점: **LLM**을 필요한 노드에서만 씁니다.

에이전트: 거의 모든 판단에 LLM (비용 ↑)

워크플로우:

[분류] → LLM (자연어 이해 필요)

[우선순위] → 규칙 (키워드 매칭이면 충분, LLM 불필요)

[배정] → 규칙 (카테고리→팀 매핑, LLM 불필요)

"분류"는 자연어 이해가 필요하니 LLM 을 쓰지만, "우선순위"나 "배정"은 간단한 규칙이면 충분합니다. LLM 없이 코드로 하면 **빠르고 공짜이고 정확**하죠. "LLM 이 잘하는 것만 LLM 에게"라는 분업입니다.

6. 무엇을 언제 쓰나

작업이 정형화돼 있고 순서가 정해졌나?

- 예 → **StateGraph** 워크플로우 (예측·검증·디버깅 쉬움)
- 아니오(탐색적·개방형) → **ReAct** 에이전트 (자유·유연)
- "티켓 분류 → 배정" 같은 정형 업무 → 워크플로우
- "이 고객 문제를 어떻게 해결할까?" 탐색 → 에이전트

그리고 실무에서는 **섞기도** 합니다 — 워크플로우의 한 노드 안에서 에이전트를 부르는 식이죠. 둘은 대립이 아니라 상황에 맞는 도구입니다.

핵심 정리

- **에이전트**: LLM 이 흐름 결정(자유·예측 불가·탐색에 강함).
- **워크플로우**: 개발자가 엣지로 흐름 고정(예측·검증·디버깅 쉬움).
- 예측 가능성은 **디버깅·감사**에 결정적(정형 업무 필수).
- 워크플로우는 **LLM 을 필요한 노드에만**(분업, 비용 ↓).
- 정형 업무는 워크플로우, 탐색은 에이전트(섞어 쓰기도).

StateGraph 의 3 요소

LangGraph 의 StateGraph 는 세 요소로 구성됩니다 — **State(상태)**: 워크플로우가 공유하는 데이터, **Node(노드)**: 상태를 읽고 일부를 채우는 함수, **Edge(엣지)**: 노드 간 실행 순서. 이 셋으로 흐름을 "선언"합니다.

1. 세 요소 한눈에

노드들이 **Edge(화살표)**로 이어지며, 각 노드가 공유 **State**를 채워 갑니다.

START → [분류 노드] → [우선순위 노드] → [배정 노드] → END

이때 모든 노드는 하나의 State `{content, category, priority, team}`를 공유하며, 각자 자기 칸을 읽고 채웁니다.

- **State**: 모든 노드가 공유하는 데이터(가운데 박스)
- **Node**: 상태를 읽고 일부를 채우는 함수(분류·우선순위·배정)
- **Edge**: 노드를 잇는 화살표(실행 순서)

2. State — 공유 상태 (TypedDict)

State는 워크플로우 전체가 공유하는 데이터입니다. `TypedDict`로 구조(스키마)를 정의합니다.

```
from typing import TypedDict

class TicketState(TypedDict):
    content: str    # 티켓 원문 (입력)
    category: str   # 분류 노드가 채움
    priority: str   # 우선순위 노드가 채움
    team: str       # 배정 노드가 채움
```

State는 "노드들이 함께 채워 가는 양식 서류"입니다. 처음엔 `content` (입력)만 있고, 노드들이 차례로 `category`, `priority`, `team`을 채웁니다. 각 노드가 한 칸씩 채워 다음 노드로 넘기는 거죠.

`TypedDict`는 "dict인데 키와 타입이 정해진 것"입니다. 일반 dict처럼 쓰지만, 어떤 키가 있어야 하는지 미리 선언하죠. 8강의 Pydantic처럼 구조를 명시하는 도구입니다.

3. Node — 상태를 채우는 함수

Node는 `state -> dict` 함수입니다. 상태를 읽고, 자기가 담당한 필드만 채워 dict로 반환합니다.

```
def classify_node(state: TicketState) -> dict:
    """티켓 내용을 카테고리 분류한다."""
    category = ... # state["content"]를 보고 분류
    return {"category": category} # 채운 필드만 반환!
```

핵심: 노드는 자기가 채운 필드만 반환합니다(`{"category": ...}`). 전체 State를 돌려줄 필요 없죠. LangGraph가 이 부분 결과를 기존 **State**에 자동 병합합니다.

`classify_node`가 `{"category": "결제"}` 반환
→ State에 `category="결제"`가 자동으로 합쳐짐 (다른 필드는 그대로)

4. Edge — 실행 순서

Edge는 노드를 잇는 화살표(실행 순서) 입니다.

```
g.add_edge(START, "classify")      # 시작 → 분류
g.add_edge("classify", "priority") # 분류 → 우선순위
g.add_edge("priority", "assign")   # 우선순위 → 배정
g.add_edge("assign", END)          # 배정 → 끝
```

`add_edge(A, B)` 는 "A 다음에 B를 실행하라"는 뜻입니다. START에서 시작해 END로 끝나죠. 이 엣지들이 흐름을 고정합니다 — 항상 분류→우선순위→배정 순서로요.

START와 **END**는 시작·종료를 나타내는 가상 노드입니다. START는 "여기서 시작", END는 "여기서 끝".

5. 핵심 API

API	역할
<code>StateGraph(State)</code>	상태 스키마로 그래프 생성
<code>add_node("name", fn)</code>	노드 등록(이름 → 함수)
<code>add_edge(A, B)</code>	A 다음에 B 실행
<code>START</code> / <code>END</code>	시작/종료 가상 노드
<code>graph.compile()</code>	실행 가능한 워크플로우로 컴파일

```
from langgraph.graph import StateGraph, START, END

g = StateGraph(TicketState)      # 1. 상태로 그래프 생성
g.add_node("classify", classify_node) # 2. 노드 등록
g.add_edge(START, "classify")    # 3. 엣지로 순서
workflow = g.compile()           # 4. 컴파일 → 실행 가능
```

`compile()`이 그래프를 **실행 가능한 워크플로우**로 만듭니다. 그러면 `workflow.invoke(초기상태)`로 실행할 수 있죠.

6. "흐름을 선언한다"

StateGraph의 철학은 "흐름을 선언적으로 표현" 하는 것입니다.

if문으로 흐름 짜기(명령형):	StateGraph로 선언하기:
result1 = classify(input)	g.add_edge(START, "classify")
result2 = priority(result1)	g.add_edge("classify", "priority")
result3 = assign(result2)	g.add_edge("priority", "assign")
→ 흐름이 코드 순서에 숨음	→ 흐름이 엣지로 한눈에 보임

`if` 문으로 짜면 흐름이 코드 곳곳에 숨지만, StateGraph는 엣지로 흐름을 명시합니다. "분류→우선 순위→배정"이 엣지 선언에 또렷이 보이죠. 복잡한 흐름(분기·반복, 06·27강)일수록 이 선언적 방식의 가치가 커집니다.

핵심 정리

- StateGraph 3 요소: **State**(공유 데이터)·**Node**(채우는 함수)·**Edge**(순서).
- **State**: TypedDict 로 정의, 노드들이 함께 채우는 양식.
- **Node**: state -> dict, **채운 필드만** 반환(자동 병합).
- **Edge**: add_edge(A, B)로 순서 고정, START~END.
- compile()로 실행 가능한 워크플로우 생성.
- 흐름을 **선언적으로** 표현(엣지로 한눈에).

따라하기 — 상태 스키마와 노드 정의

목표

티켓 처리 워크플로우의 상태(TypedDict)와 세 노드(분류·우선순위·배정)를 만듭니다. 분류만 LLM, 나머지는 규칙으로요. 실제 파일은 `code/ch21_workflow.py` 입니다.

0. 실습 준비

데이터는 `data/support_tickets.csv(ticket_id, content, ...)`를 씁니다.

```
conda activate agentic
python code/common.py
```

1. 공통 시작부 + 상태 스키마

```
import sys, pathlib
sys.path.append(str(pathlib.Path(__file__).resolve().parent))
from common import get_chat, DATA
import csv
from typing import TypedDict
from langchain_core.messages import SystemMessage, HumanMessage
from langgraph.graph import StateGraph, START, END

class TicketState(TypedDict):
    # 모든 노드가 공유하는 상태. 각 노드는 일부 필드만 채워 dict로 반환하면 자동 병합
    content: str    # 티켓 원문 (입력)
    category: str   # 분류 노드가 채움
    priority: str   # 우선순위 노드가 채움
    team: str       # 배정 노드가 채움
```

State에 4개 필드를 선언했습니다. `content`는 입력이고, 나머지 셋은 노드들이 차례로 채울 빈칸이죠.

2. 노드 1 — 분류 (LLM)

자연어 이해가 필요하니 **LLM**을 씁니다.

```
llm = get_chat(temperature=0)
CATEGORIES = ["결제", "배송", "환불", "교환", "회원", "기술지원", "기타"]

def classify_node(state: TicketState) -> dict:
    """[노드1] 티켓 내용을 카테고리 분류한다. (LLM)"""
    msg = [
        SystemMessage(f"다음 CS 티켓을 {CATEGORIES} 중 하나로만 분류하라. 카테고리 단어만 출력."),
        HumanMessage(state["content"]),
    ]
    out = llm.invoke(msg).text.strip()
    category = next((c for c in CATEGORIES if c in out), "기타") # 목록에 없으면 '기타' 폴백
    return {"category": category} # 채운 필드만 반환 → 상태에 자동 병합
```

뜯어보면:

- state["content"]로 티켓 내용을 읽음
- LLM 으로 7 개 카테고리 중 하나로 분류
- next((c for c in CATEGORIES if c in out), "기타"): 결과가 목록에 없으면 "기타"로 폴백
- {"category": category}만 반환 → State 에 자동 병합

temperature=0 으로 **분류가 일관되게** 합니다(같은 티켓 → 같은 분류).

3. 노드 2 — 우선순위 (규칙)

키워드 규칙이면 충분하니 **LLM 없이** 처리합니다.

```
URGENT_WORDS = ["결제", "오류", "안 돼", "파손", "안와", "안 와"]
HIGH_WORDS = ["환불", "교환", "팅"]

def priority_node(state: TicketState) -> dict:
    """[노드2] 카테고리/키워드로 우선순위를 정한다. (규칙)"""
    text = state["content"]
```

```

if state["category"] == "결제" or any(w in text for w in URGENT_WORDS):
    priority = "긴급"
elif state["category"] in ("환불", "교환") or any(w in text for w in HIGH_WORDS):
    priority = "높음"
else:
    priority = "보통"
return {"priority": priority}

```

이 노드는 **LLM을 안 씁니다**. 키워드 매칭(`any(w in text ...)`)이면 충분하거든요. 빠르고 공짜이고 정확하죠(02번 분업). "결제 카테고리거나 긴급 단어가 있으면 긴급" 같은 규칙을 코드로 명확히 표현합니다.

`state["category"]`를 읽는 게 포인트입니다 — 앞 노드(분류)가 채운 값을 이 노드가 활용하죠. State를 통해 노드들이 협력하는 모습입니다.

4. 노드 3 — 배정 (규칙)

카테고리 → 팀 매핑도 규칙이면 충분합니다.

```

TEAM_MAP = {
    "결제": "결제지원팀", "환불": "정산팀", "교환": "물류팀",
    "배송": "물류팀", "회원": "회원관리팀", "기술지원": "기술지원팀",
}

def assign_node(state: TicketState) -> dict:
    """[노드3] 카테고리에 맞는 담당팀을 배정한다. (규칙)"""
    team = TEAM_MAP.get(state["category"], "일반상담팀") # 없으면 일반상담팀
    return {"team": team}

```

딕셔너리 매핑 한 줄이면 끝입니다. "결제→결제지원팀" 같은 매핑은 LLM이 필요 없죠. `.get(..., "일반상담팀")`으로 매핑에 없는 카테고리는 "일반상담팀"으로 풀백합니다.

5. 노드의 공통 패턴

세 노드를 보면 공통 패턴이 있습니다.

```
def 노드(state: TicketState) -> dict:
    값 = state["입력_필드"]를 읽어서 처리
    return {"출력_필드": 결과} # 채운 것만 반환
```

- 입력: `state` 에서 필요한 필드를 읽음
- 처리: LLM 또는 규칙으로 계산
- 출력: 자기가 채운 필드만 dict로 반환

이 패턴이 모든 노드의 골격입니다. 노드는 작고 단일 책임을 가지죠 — 분류 노드는 분류만, 배정 노드는 배정만. 그래서 디버깅이 쉽습니다.

핵심 정리

- State 는 TypedDict 로 4 필드(content + 채울 3 개) 선언.
- **분류 노드는 LLM**(자연어 이해), **우선순위·배정은 규칙**(분업).
- 각 노드는 state 읽고 **채운 필드만** 반환(자동 병합).
- 뒷 노드가 앞 노드의 결과(state["category"])를 활용.
- 노드는 작고 **단일 책임**(디버깅 쉬움).

따라하기 — 그래프 조립과 일괄 처리

목표

04번의 세 노드를 StateGraph로 조립하고, CSV의 티켓들을 일괄 처리합니다. 같은 입력이면 항상 같은 결과가 나오는 결정적 워크플로우를 확인합니다.

1. 그래프 조립

노드를 등록하고 엣지로 순서를 고정합니다.

```

from langgraph.graph import StateGraph, START, END

def build_workflow():
    g = StateGraph(TicketState)          # 상태 스키마로 그래프 생성
    g.add_node("classify", classify_node) # 노드 등록(이름 → 함수)
    g.add_node("priority", priority_node)
    g.add_node("assign", assign_node)
    # 엣지로 실행 순서를 '고정'한다. 결정적 흐름: 분류 → 우선순위 → 배정
    g.add_edge(START, "classify")
    g.add_edge("classify", "priority")
    g.add_edge("priority", "assign")
    g.add_edge("assign", END)
    return g.compile()                  # 실행 가능한 워크플로우로 컴파일

```

네 단계입니다.

1. `StateGraph(TicketState)`: 상태로 그래프 생성
2. `add_node`: 세 노드 등록
3. `add_edge`: START→분류→우선순위→배정→END 순서 고정
4. `compile()`: 실행 가능하게 변환

엣지 4 줄이 **흐름을 한눈에** 보여 줍니다 — "분류 다음 우선순위, 그 다음 배정".
이게 "선언적 흐름"입니다.

2. 워크플로우 실행

`invoke`에 초기 상태(content)만 넣으면, 세 노드가 차례로 실행됩니다.

```

workflow = build_workflow()
result = workflow.invoke({"content": "결제가 안 돼요. 계속 오류가 납니다."})
print(result)

```


예상 출력

```
{
  "content": "결제가 안 돼요. 계속 오류가 납니다.",
  "category": "결제",    ← classify_node가 채움
  "priority": "긴급",    ← priority_node가 채움
  "team": "결제지원팀"  ← assign_node가 채움
}
```

처음에 `content` 만 넣었는데, 워크플로우를 거치며 `category`, `priority`, `team` 이 차례로 채워졌습니다. 세 노드가 State를 함께 완성한 거죠.

3. State가 채워지는 과정

`invoke({"content": "결제가 안 돼요"})` 가 노드를 거치며 State가 채워집니다.

1. **START** → **classify** → `{content: "결제가 안 돼요", category: "결제"}` (*category 추가*)
2. **priority** → `{..., category: "결제", priority: "긴급"}` (*priority 추가*)
3. **assign** → `{..., priority: "긴급", team: "결제지원팀"}` (*team 추가*)
4. **END** → 최종 State 반환

각 노드가 State에 한 칸씩 채워 넣는 게 보입니다. 빈 양식(content만)이 노드들을 거치며 완성된 양식(전부 채워짐)이 되죠.

4. CSV 일괄 처리

여러 티켓을 한 번에 처리합니다.

```
with open(DATA / "support_tickets.csv", encoding="utf-8-sig") as f:
    tickets = list(csv.DictReader(f))

print(f"{'티켓':<7}{ '분류':<7}{ '우선순위':<7}{ '담당팀':<10}내용")
print("-" * 70)
for t in tickets:
    result = workflow.invoke({"content": t["content"]})  # 한 티켓 처리
    print(f"{t['ticket_id']:<7}{result['category']:<7}"
          f"{result['priority']:<7}{result['team']:<10}{t['content'][:20]}")
```

예상 출력

티켓	분류	우선순위	담당팀	내용
TK001	결제	긴급	결제지원팀	결제가 안 돼요. 계속 오류가...
TK002	기타	보통	일반상담팀	상품 색상 문의드립니다.
TK003	배송	긴급	물류팀	배송이 일주일째 안 와요...
TK005	교환	높음	물류팀	받은 상품이 파손되어 왔습니다...
TK007	기술지원	긴급	기술지원팀	앱이 자주 튕겨요.

각 티켓이 분류→우선순위→배정을 거쳐 깔끔하게 처리됐습니다. `utf-8-sig`는 엑셀 CSV의 BOM(보이지 않는 문자)을 제거해 읽는 인코딩입니다.

5. 결정적 — 항상 같은 결과

이 워크플로우의 핵심 특징: 같은 입력이면 항상 같은 결과입니다.

"결제가 안 돼요" → 항상 결제/긴급/결제지원팀
(분류 LLM은 `temperature=0`, 우선순위·배정은 규칙)

분류 LLM은 `temperature=0`이라 일관되고, 우선순위·배정은 규칙이라 결정적이죠. 그래서 업무 규칙(분류→우선순위→배정)이 코드로 강제되고, 검증·감사가 쉽습니다(02번). "이 티켓이 왜 이 팀에 갔지?"를 노드 단위로 추적할 수 있죠.

6. 배치 잡으로 확장

이 워크플로우는 그대로 배치 잡이나 큐 컨슈머로 감쌀 수 있습니다.

들어온 티켓마다 → `workflow.invoke({"content": ...})`
→ 결과(team)에 따라 메시지 큐로 라우팅
→ 노드 단위라 단계별 로깅·예외처리·재시도 깔끔

실시간으로 들어오는 티켓을 큐에서 꺼내 워크플로우에 넣고, 결과 팀으로 보내는 식이죠. 노드가 작게 나뉘어 있어 운영에 붙이기 좋습니다(07번 예외처리).

핵심 정리

- `add_node` + `add_edge` 로 그래프 조립, `compile()`로 완성.
- `invoke({"content": ...})`에 초기 상태만 넣으면 전체 실행.
- 노드들이 State 를 한 칸씩 채워 완성.
- 같은 입력 → 항상 같은 결과(결정적, 검증·감사 쉬움).
- 그대로 배치 잡·큐 컨슈머로 확장 가능.

조건부 분기 (`add_conditional_edges`)

지금까지는 노드들이 일직선으로 흘렀습니다. 하지만 "긴급하면 에스컬레이션, 아니면 일반 배정"처럼 상황에 따라 길이 갈려야 할 때가 있습니다. `add_conditional_edges` 로 상태를 보고 다음 노드를 고르는 갈림길(분기) 을 만듭니다.

1. 일직선의 한계

05번 워크플로우는 모든 티켓이 같은 길을 갔습니다.

모든 티켓: 분류 → 우선순위 → 배정 → 끝 (일직선)

하지만 현실은 다릅니다. 긴급 티켓은 일반 배정 큐를 거치면 늦죠.

- 긴급 티켓: 분류 → 우선순위 → 즉시 에스컬레이션! → 끝
- 일반 티켓: 분류 → 우선순위 → 일반 배정 → 끝

우선순위 단계 뒤에서 길이 갈려야 합니다.

우선순위가 "긴급"이냐 아니냐에 따라 다른 노드로 가야 합니다. 이게 조건부 분기입니다.

2. `add_conditional_edges` — 갈림길

`add_conditional_edges` 는 상태를 보고 다음 노드를 동적으로 고릅니다. (실습 파일:

`sections/21_agent_workflow/06_조건부-분기-add_conditional_edges.py`)

```
def route_by_priority(state: TicketState) -> str:
    """우선순위를 보고 다음 노드 이름을 반환하는 라우터 함수."""
    return "escalate" if state["priority"] == "긴급" else "assign"
```

이게 라우터 함수입니다. State를 받아 "다음에 갈 노드 이름"을 문자열로 돌려주죠. "긴급이면 escalate, 아니면 assign"이라고요. `add_conditional_edges`가 이 반환값으로 다음 노드를 정합니다.

3. 분기 노드 만들기

두 갈래의 노드를 만듭니다.

```
def escalate_node(state: TicketState) -> dict:
    """[긴급 분기] 즉시 상위팀으로 에스컬레이션한다."""
    return {
        "team": "긴급대응팀",
        "route": "긴급에스컬",
        "action": "SLA 30분 내 즉시 에스컬레이션(담당자 호출)",
    }

def assign_route_node(state: TicketState) -> dict:
    """[일반 분기] 카테고리에 맞는 담당팀으로 일반 배정한다."""
    upd = assign_node(state) # 기존 배정 규칙 재사용
    upd["route"] = "일반배정"
    upd["action"] = f"{upd['team']} 일반 배정(SLA 24시간)"
    return upd
```

`escalate_node`는 긴급용(30분 SLA), `assign_route_node`는 일반용(24시간 SLA)입니다. 긴급은 빠른 경로로, 일반은 보통 경로로 가는 거죠.

4. 조건부 엣지 연결

`add_conditional_edges`로 분기를 그래프에 연결합니다.

```
g.add_edge(START, "classify")
g.add_edge("classify", "priority")
```

```
# 선형 엣지 대신 조건부 분기: priority 다음은 라우터 함수가 정한다
g.add_conditional_edges(
    "priority",                                # 이 노드 다음에
    route_by_priority,                          # 라우터 함수가 다음을 정함
    {"escalate": "escalate", "assign": "assign"}, # 반환값 → 실제 노드 매핑
)
g.add_edge("escalate", END)
g.add_edge("assign", END)
```

`add_conditional_edges(노드, 라우터, 매핑)`:

- 노드: `"priority"` — 이 노드 다음에 분기
- 라우터: `route_by_priority` — 다음 노드 이름을 결정하는 함수
- 매핑: `{"escalate": "escalate", ...}` — 라우터 반환값을 실제 노드로 연결

이러면 priority 노드 다음에 라우터가 작동해, 긴급이면 escalate로, 아니면 assign으로 갑니다.

5. 흐름 그림

- START → **[classify]** → **[priority]** → `route_by_priority` (라우터)
- 라우터가 우선순위를 보고 갈림:
- 긴급? → **[escalate]** → END
- 아니면 → **[assign]** → END

priority 다음에 갈림길이 생겼습니다. 라우터가 우선순위를 보고 둘 중 하나로 보내죠.

6. 실행 결과

```
for t in tickets:
    result = workflow.invoke({"content": t["content"]})
    print(f"{t['ticket_id']} {result['priority']} {result['route']} {result['team']} {result['action']}"
```

예상 출력

TK001	긴급	긴급에스컬	긴급대응팀	SLA 30분 내 즉시 에스컬레이션
TK002	보통	일반배정	일반상담팀	일반상담팀 일반 배경(SLA 24시간)
TK003	긴급	긴급에스컬	긴급대응팀	SLA 30분 내 즉시 에스컬레이션
TK005	높음	일반배정	물류팀	물류팀 일반 배경(SLA 24시간)

긴급 티켓(TK001, TK003)은 에스컬레이션 경로로, 나머지는 일반 배경 경로로 갔습니다. 같은 워크플로우인데 우선순위에 따라 다른 길을 탄 거죠.

7. 조건부 분기의 가치

조건부 분기로 워크플로우가 훨씬 유연해집니다.

선형(05번): 모든 케이스가 같은 처리 (단순하지만 경직)
분기(06번): 케이스별 다른 처리 (긴급↔일반, 정상↔오류 등)

분기를 쓰면 "긴급은 빠르게, 일반은 보통대로", "정상은 진행, 오류는 재시도" 같은 **상황별 처리**가 가능합니다. 분기로 "품질 부족하면 되돌아가기" 같은 루프도 만들죠. 분기는 워크플로우를 진짜 업무 흐름에 가깝게 만듭니다.

핵심 정리

- 일직선 워크플로우는 **상황별 처리**를 못 함.
- **add_conditional_edges** 로 상태를 보고 다음 노드를 동적 선택.
- **라우터 함수**(state -> 노드이름)가 분기를 결정.
- add_conditional_edges(노드, 라우터, 매핑)으로 연결.
- 긴급↔일반, 정상↔오류 같은 **상황별 처리** 가능.

노드 실패 처리와 상태 검증

LLM 분류 결과가 허용 카테고리 밖이면 '기타'로 보정하고, 예외가 나도 죽지 않도록 `state["error"]`에 기록한 뒤 파이프라인을 계속 진행합니다. 한 티켓이 실패해도 전체 배치가 멈추지 않는 견고한 워크플로우를 만듭니다.

1. 워크플로우도 실패한다

지금까지는 모든 게 잘 되는 경우(happy path)만 봤습니다. 하지만 현실에서는:

- LLM이 허용 밖 카테고리를 반환 ("음... 아마 결제 관련?")
- LLM API 호출 실패 (네트워크 · 타임아웃)
- 빈 티켓 내용 (입력 오류)

이런 일이 생기면 어떻게 될까요? 처리 안 하면 한 티켓의 실패가 전체 배치를 멈출 수 있습니다. 1000개 티켓을 처리하다 500번째에서 죽으면 큰일이죠.

2. 상태에 error 필드 추가

먼저 State에 오류를 기록할 필드를 추가합니다. (실습 파일: [sections/21_agent_workflow/07_노드-실패-처리와-상태-검증.py](#))

```
class TicketState(BaseState): # 기존 상태 확장
    error: str # 오류 메시지(비어 있으면 정상)
```

`error`가 비어 있으면 정상, 뭔가 있으면 문제가 있었다는 뜻이죠. 이걸로 "어느 티켓이 어떤 문제였는지" 추적할 수 있습니다.

3. 검증 — LLM 출력을 믿지 마라

LLM 분류 결과를 검증합니다. 허용 카테고리 안에 있을 때만 채택하고, 아니면 보정하죠.

```
def classify_node(state: TicketState) -> dict:
    """[견고화] 분류 결과를 '검증'하고, 예외가 나도 멈추지 않는다."""
    try:
        out = llm.invoke(msg).content.strip()
        # LLM 출력은 못 믿는다 → 허용 목록 안에 있을 때만 채택
        if out in CATEGORIES:
            return {"category": out, "error": ""}
        for c in CATEGORIES:          # 문장 속에 카테고리가 섞인 경우도 구제
            if c in out:
                return {"category": c, "error": ""}
        # 검증 실패: 허용 밖 값 → '기타'로 보정하고 사유 기록 (중단 안 함)
        return {"category": "기타", "error": f"분류값 미허용('{out[:20]}') → 기타 보정"}
    except Exception as e:
        # 한 티켓의 LLM 호출 실패해도 전체 배치가 죽으면 안 됨 → 기록 후 계속
        return {"category": "기타", "error": f"분류 예외: {e}"}
```

세 단계 방어:

1. 허용 목록 검증: `if out in CATEGORIES` — 정확히 맞으면 채택
2. 부분 매칭 구제: 문장에 카테고리가 섞여 있으면("아마 결제...") 추출
3. 보정 + 기록: 그래도 안 되면 "기타"로 보정하고 사유를 `error`에 기록

핵심은 "LLM 출력을 그냥 믿지 않는다" 입니다(16강 환각 정신). 검증해서 이상하면 안전한 기본 값("기타")으로 보정하죠.

4. 예외도 잡는다

`try/except`로 LLM 호출 자체가 실패해도(네트워크 등) 죽지 않게 합니다.

```
except Exception as e:
    # 예외가 나도 '기타'로 보정하고 사유 기록 → 파이프라인 계속
    return {"category": "기타", "error": f"분류 예외: {e}"}
```


예외가 나도 **return** 으로 **정상 반환**합니다(예외를 다시 던지지 않음). 그러면 워크플로우가 멈추지 않고 다음 노드로 진행하죠(5.12 강 폴백 정신). 한 티켓의 실패가 전체를 멈추지 않게 하는 겁니다.

5. 죽지 않음을 시연

일부러 깨진 입력(빈 내용)을 끼워 넣어 죽지 않음을 보여 줍니다.

```
# 의도적으로 깨진 입력을 끼워 넣어 '죽지 않음'을 시연
tickets.append({"ticket_id": "TKBAD", "content": ""})

for t in tickets:
    result = workflow.invoke({"content": t["content"], "error": ""})
    flag = result.get("error") or "정상"
    print(f"{t['ticket_id']} {result['category']} {result['priority']} {result['team']} {flag}")
```

예상 출력

TK001	결제	긴급	결제지원팀	정상
TK002	기타	보통	일반상담팀	정상
TKBAD	기타	보통	일반상담팀	분류값 미허용('') → 기타 보정 ← 죽지 않고 처리!

빈 티켓(TKBAD)도 죽지 않고 처리됐습니다. "기타/일반상담팀"으로 보정되고, **error**에 사유가 기록됐죠. 전체 배치는 끝까지 완주했습니다.

6. 노드 단위 견고성의 장점

워크플로우가 노드로 나뉘어 있어, 어느 단계에서 실패했는지 명확합니다.

```
error: "분류 예외: ..." → classify_node에서 실패
error: "우선순위 예외: ..." → priority_node에서 실패
```

`error` 메시지만 봐도 어느 노드 문제인지 즉시 압니다. 실무에서는:

- 로깅: 실패한 티켓을 별도 로그에 기록 → 나중에 분석
- 재시도: 일시적 실패면 그 노드만 다시 실행
- 알람: 실패율이 높으면 담당자에게 경고
- 수동 검토 큐: `error`가 있는 티켓은 "검토필요" 큐로

노드 단위라 이런 운영 장치를 깔끔히 붙일 수 있죠(02번).

7. 견고함이 진짜 서비스의 조건

happy path만 처리하는 워크플로우는 장난감입니다. 진짜 서비스는:

정상 처리(99%)도 잘하고
실패(1%)도 우아하게 다뤄야 함 (죽지 않고, 기록하고, 복구)

1000 개 중 999 개가 잘 돼도, 1 개에서 죽어 배치 전체가 멈추면 서비스 실패죠. "검증 + 예외 처리 + 기록"으로 견고하게 만드는 게 운영 가능한 워크플로우의 조건입니다.

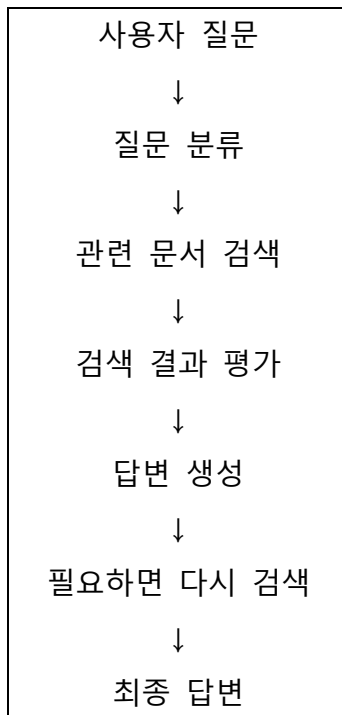
핵심 정리

- 워크플로우도 **실패**한다(허용 밖 분류·예외·빈 입력).
- State 에 **error** 필드를 뒤 문제를 기록.
- **검증**: LLM 출력을 믿지 말고 허용 목록 확인, 이상하면 보정.
- **예외 처리**: try/except 로 죽지 않고 기록 후 계속.
- 노드 단위라 **어느 단계 실패인지** 명확(로깅·재시도·검토 큐).
- 견고함이 **진짜 서비스의 조건**.

2. LangGraph API

LangGraph 는 LLM, 도구, 데이터 처리 함수 등을 노드(Node)로 만들고, 공유 상태(State)와 실행 경로(Edge)를 연결하여 복잡한 AI 에이전트와 워크플로우를 구현하는 Python 프레임워크입니다.

단순히 LLM 을 한 번 호출하는 것이 아니라 다음과 같은 장시간 실행 구조를 만들 때 사용합니다.



LangGraph 는 특히 상태 유지, 조건부 분기, 반복 실행, 중단 후 재개, 사람 승인, 멀티에이전트 협업 같은 기능을 직접 제어해야 하는 경우에 적합합니다. 공식 문서는 LangGraph 를 장시간 실행되는 상태 기반 에이전트를 위한 저수준 오케스트레이션 프레임워크로 설명합니다. ([GitHub](#))

1. LangGraph 가 필요한 이유

1.1 일반적인 LLM 호출

가장 단순한 LLM 애플리케이션은 질문을 전달하고 답변을 받습니다.

사용자 질문 → LLM → 답변

Python 코드로 표현하면 다음과 비슷합니다.

```
# 사용자가 입력한 질문을 LLM 에 전달합니다.
response = llm.invoke("환불 기간을 알려주세요.")

# LLM 이 생성한 답변을 출력합니다.
print(response.content)
```

이 방식은 간단하지만 다음과 같은 복잡한 작업을 처리하기 어렵습니다.

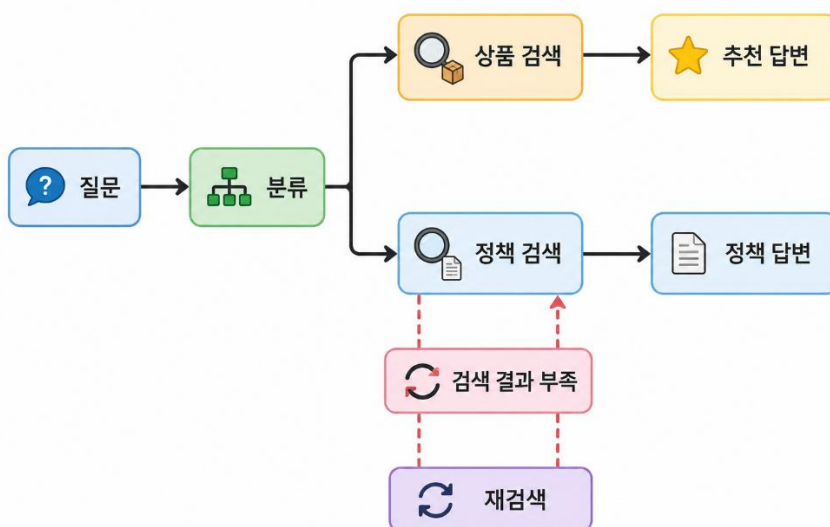
- 질문 종류에 따라 다른 기능 실행
- 검색 결과가 부족하면 다시 검색
- 여러 도구 중 적절한 도구 선택
- 중간 처리 결과 저장
- 실패한 단계부터 다시 실행
- 사용자 승인을 받은 후 다음 단계 진행
- 여러 전문 에이전트가 역할을 나누어 처리

1.2 체인 방식의 한계

LangChain 의 일반적인 체인은 작업을 순서대로 연결할 수 있습니다.

입력 → 프롬프트 → LLM → 출력 파서

그러나 실행 흐름이 다음처럼 복잡해지면 단순한 직선 체인만으로는 관리하기 어렵습니다.



여기에는 다음 기능이 필요합니다.

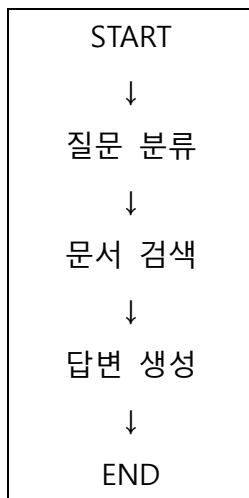
- 분기
- 반복
- 상태 공유
- 병렬 실행
- 오류 복구
- 실행 중단
- 실행 재개

LangGraph 는 이런 흐름을 그래프 구조로 표현합니다.

2. Workflow 와 Agent 의 차이

2.1 Workflow

워크플로우는 개발자가 실행 순서를 미리 정합니다.

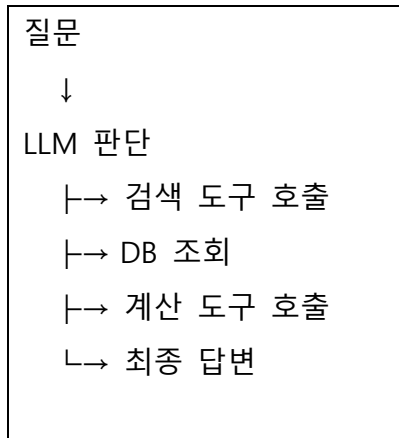


특징은 다음과 같습니다.

항목	설명
실행 순서	개발자가 정의
예측 가능성	높음
테스트	단계별 테스트가 쉬움
적합한 작업	정해진 업무 절차
예	상담 분류, 문서 검증, 승인 처리

2.2 Agent

에이전트는 실행 중 LLM 이 다음 행동을 결정합니다.



특징은 다음과 같습니다.

항목	설명
실행 순서	LLM 이 동적으로 결정
예측 가능성	워크플로우보다 낮음
유연성	높음
적합한 작업	탐색적이고 개방적인 업무
예	리서치 에이전트, 코딩 에이전트

공식 문서에서도 Workflow 는 미리 정해진 코드 경로를 따르고, Agent 는 자신의 처리 과정과 도구 사용을 동적으로 결정하는 구조로 구분합니다. ([Docs by LangChain](#))

3. LangGraph 의 핵심 3 요소

LangGraph 의 Graph API 는 다음 세 가지를 중심으로 구성됩니다.

State + Node + Edge

공식 문서에서도 State 는 공유 데이터, Node 는 처리 로직, Edge 는 다음에 실행할 노드를 결정하는 연결로 정의합니다. ([Docs by LangChain](#))

3.1 State — 공유 상태

State 는 그래프 전체에서 공유되는 데이터입니다.

예를 들어 고객 문의를 처리하는 그래프라면 다음 정보를 상태로 관리할 수 있습니다.

```
# TypedDict 를 사용하여 상태에 들어갈 키와 값의 타입을 정의합니다.
from typing import TypedDict

class SupportState(TypedDict):
    # 사용자가 입력한 원본 질문을 저장합니다.
    question: str

    # 질문을 분류한 결과를 저장합니다.
    category: str

    # Vector DB 에서 검색한 문서를 저장합니다.
    documents: list[str]

    # LLM 이 생성한 최종 답변을 저장합니다.
    answer: str

    # 처리 중 발생한 오류 메시지를 저장합니다.
    error: str
```

처음 상태는 다음처럼 일부 값만 존재할 수 있습니다.

```
initial_state = {
    "question": "환불은 며칠 이내에 가능한가요?",
    "category": "",
    "documents": [],
    "answer": "",
    "error": "",
}
```

노드가 실행될 때마다 상태가 갱신됩니다.

초기 상태

```
{
  question: "환불 기간은?",
  category: "",
  documents: [],
  answer: ""
}
```

분류 노드 실행 후

```
{
  question: "환불 기간은?",
  category: "policy",
  documents: [],
  answer: ""
}
```

검색 노드 실행 후

```
{
  question: "환불 기간은?",
  category: "policy",
  documents: ["상품 수령 후 7 일 이내 신청 가능합니다."],
  answer: ""
}
```

답변 노드 실행 후

```
{
  question: "환불 기간은?",
  category: "policy",
  documents: ["상품 수령 후 7 일 이내 신청 가능합니다."],
  answer: "환불은 상품 수령 후 7 일 이내에 신청할 수 있습니다."
}
```


3.2 Node — 작업을 수행하는 함수

Node 는 현재 State 를 입력으로 받아 작업을 수행하고, 변경할 상태 값만 반환하는 함수입니다.

```
def classify_node(state: SupportState) -> dict:
    """
    사용자의 질문을 분석하여 정책 질문인지 상품 추천 질문인지 분류합니다.

    매개변수:
        state:
            그래프가 현재 보유하고 있는 전체 상태입니다.

    반환값:
        category 키만 포함한 딕셔너리를 반환합니다.
        LangGraph 가 이 결과를 기존 상태에 병합합니다.
    """

    # 상태에서 사용자의 질문을 가져옵니다.
    question = state["question"]

    # 정책 관련 핵심 단어를 목록으로 정의합니다.
    policy_words = ["환불", "교환", "배송", "취소", "적립", "회원"]

    # 질문에 정책 관련 단어가 하나라도 포함되어 있는지 검사합니다.
    is_policy = any(word in question for word in policy_words)

    # 정책 단어가 있으면 policy, 없으면 sales 로 분류합니다.
    category = "policy" if is_policy else "sales"

    # 전체 상태가 아니라 이 노드가 변경한 값만 반환합니다.
    return {"category": category}
```

Node 의 핵심 규칙은 다음과 같습니다.

입력 : 현재 State

처리 : LLM 호출, DB 조회, 계산, API 호출 등

출력 : 변경할 State 값

Node 안에는 반드시 LLM 이 들어갈 필요가 없습니다.

Node 가 할 수 있는 일:

- 일반 Python 계산
- 규칙 기반 분류
- OpenAI API 호출
- Gemini API 호출
- MySQL 조회
- Vector DB 검색
- MCP Tool 호출
- 외부 REST API 호출
- 파일 읽기

공식 문서도 Node 와 Edge 는 결국 일반 함수이며, Node 내부에는 LLM 뿐 아니라 일반 코드도 넣을 수 있다고 설명합니다. ([Docs by LangChain](#))

3.3 Edge — 실행 순서를 연결

Edge 는 어떤 Node 다음에 어떤 Node 를 실행할지 정의합니다.

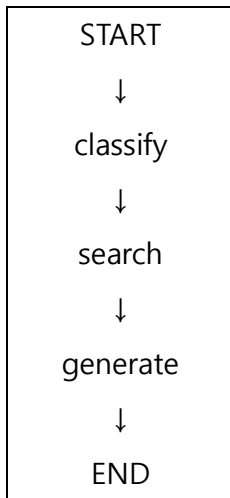
```
# START 다음에 classify 노드를 실행합니다.
graph_builder.add_edge(START, "classify")

# classify 노드 다음에 search 노드를 실행합니다.
graph_builder.add_edge("classify", "search")

# search 노드 다음에 generate 노드를 실행합니다.
graph_builder.add_edge("search", "generate")

# generate 노드가 끝나면 그래프를 종료합니다.
graph_builder.add_edge("generate", END)
```

그래프는 다음 흐름으로 실행됩니다.



4. LangGraph 기본 설치

4.1 패키지 설치

```
python -m pip install --upgrade pip setuptools wheel  
pip install -U langgraph
```

OpenAI 를 함께 사용할 경우 다음 패키지를 설치합니다.

```
pip install -U langchain-openai
```

Gemini 를 함께 사용할 경우 다음 패키지를 설치합니다.

```
pip install -U langchain-google-genai
```

환경변수 파일을 사용하려면 다음 패키지도 설치합니다.

```
pip install -U python-dotenv
```

5. 가장 기본적인 StateGraph

5.1 전체 코드

```
# -*- coding: utf-8 -*-

"""
LangGraph 기본 실행 예제

실행 흐름:
START → input_node → process_node → output_node → END
"""

# TypedDict 는 딕셔너리의 키와 값 타입을 선언할 때 사용합니다.
from typing import TypedDict

# StateGraph 는 상태 기반 그래프를 생성하는 핵심 클래스입니다.
# START 는 그래프의 시작점을 나타냅니다.
# END 는 그래프의 종료점을 나타냅니다.
from langgraph.graph import StateGraph, START, END


class BasicState(TypedDict):
    """
    모든 노드가 공유할 상태 구조를 정의합니다.
    """

    # 사용자가 입력한 원본 문장을 저장합니다.
    text: str

    # 문자열의 길이를 저장합니다.
    length: int

    # 최종 처리 결과를 저장합니다.
```

```
result: str
```

```
def input_node(state: BasicState) -> dict:
```

```
    """
```

```
    입력 문장을 읽고 문자열 길이를 계산합니다.
```

```
    """
```

```
    # 현재 상태에서 text 값을 가져옵니다.
```

```
    text = state["text"]
```

```
    # Python 의 len 함수를 사용하여 문자열 길이를 계산합니다.
```

```
    text_length = len(text)
```

```
    # 계산된 길이만 상태 업데이트 값으로 반환합니다.
```

```
    return {"length": text_length}
```

```
def process_node(state: BasicState) -> dict:
```

```
    """
```

```
    입력 문장과 계산된 길이를 이용해 결과 문자열을 생성합니다.
```

```
    """
```

```
    # 앞 노드가 저장한 입력 문장을 읽습니다.
```

```
    text = state["text"]
```

```
    # 앞 노드가 계산한 문자열 길이를 읽습니다.
```

```
    length = state["length"]
```

```
    # 사용자에게 보여 줄 결과 문장을 생성합니다.
```

```
    result = f"입력 문장: {text} / 문자 수: {length}"
```

```
    # 생성한 결과만 상태에 반영하도록 반환합니다.
```

```

    return {"result": result}

def output_node(state: BasicState) -> dict:
    """
    최종 결과를 콘솔에 출력합니다.
    """

    # 현재 상태의 최종 결과를 출력합니다.
    print(state["result"])

    # 이 노드는 상태를 변경하지 않으므로 빈 딕셔너리를 반환합니다.
    return {}

# BasicState 를 공유 상태로 사용하는 그래프 빌더를 생성합니다.
graph_builder = StateGraph(BasicState)

# input_node 함수를 input 이라는 이름으로 등록합니다.
graph_builder.add_node("input", input_node)

# process_node 함수를 process 라는 이름으로 등록합니다.
graph_builder.add_node("process", process_node)

# output_node 함수를 output 이라는 이름으로 등록합니다.
graph_builder.add_node("output", output_node)

# START 다음에 input 노드를 실행합니다.
graph_builder.add_edge(START, "input")

# input 다음에 process 노드를 실행합니다.
graph_builder.add_edge("input", "process")

```

```

# process 다음에 output 노드를 실행합니다.
graph_builder.add_edge("process", "output")

# output 노드 실행 후 그래프를 종료합니다.
graph_builder.add_edge("output", END)

# 정의한 그래프를 실제 실행 가능한 객체로 컴파일합니다.
workflow = graph_builder.compile()

# 그래프에 전달할 초기 상태를 생성합니다.
initial_state = {
    "text": "LangGraph 를 학습합니다.",
    "length": 0,
    "result": "",
}

# invoke 를 사용해 그래프를 동기 방식으로 실행합니다.
final_state = workflow.invoke(initial_state)

# 모든 노드 실행이 끝난 후의 최종 상태를 출력합니다.
print(final_state)

```

5.2 실행 결과

```

입력 문장: LangGraph 를 학습합니다. / 문자 수: 16

{
  'text': 'LangGraph 를 학습합니다.',
  'length': 16,
  'result': '입력 문장: LangGraph 를 학습합니다. / 문자 수: 16'
}

```

6. compile 과 invoke

6.1 compile()

그래프에 Node 와 Edge 를 등록했다고 바로 실행할 수 있는 것은 아닙니다.

```
workflow = graph_builder.compile()
```

compile()은 다음 사항을 확인하고 실행 가능한 그래프로 변환합니다.

- 시작점이 있는가?
- 연결되지 않은 노드는 없는가?
- 실행 경로가 올바른가?
- 상태 구조를 사용할 수 있는가?
- 체크포인트가 설정됐는가?

공식 Quickstart 도 StateGraph 를 구성한 후 compile()을 호출해 실행 가능한 에이전트로 만듭니다. ([Docs by LangChain](#))

6.2 invoke()

invoke()는 입력 상태를 전달하고 그래프를 한 번 실행합니다.

```
result = workflow.invoke(  
    {  
        "text": "LangGraph 실행",  
        "length": 0,  
        "result": "",  
    }  
)
```

반환값은 모든 노드 실행이 끝난 최종 State 입니다.

6.3 ainvoke()

FastAPI 처럼 비동기 환경에서는 ainvoke()를 사용할 수 있습니다.

```
result = await workflow.ainvoke(  
    {  
        "text": "비동기 LangGraph 실행",  
        "length": 0,  
        "result": "",  
    }  
)
```

FastAPI 엔드포인트 예시는 다음과 같습니다.

```
from fastapi import APIRouter  
  
router = APIRouter()  
  
@router.post("/graph/run")  
async def run_graph(text: str):  
    """  
    FastAPI 에서 LangGraph 를 비동기 방식으로 실행합니다.  
    """  
    # await 를 사용하여 비동기 그래프를 실행합니다.  
    result = await workflow.ainvoke(  
        {  
            "text": text,  
            "length": 0,  
            "result": "",  
        }  
    )  
  
    # 최종 상태를 HTTP 응답으로 반환합니다.  
    return result
```

7. 조건부 분기

7.1 조건부 분기가 필요한 이유

모든 입력을 같은 순서로 처리할 필요는 없습니다.

사용자 질문

↓

질문 분류

└→ 상품 질문 → 상품 검색

└→ 정책 질문 → 정책 검색

이때 `add_conditional_edges()`를 사용합니다.

7.2 조건부 분기 코드

```
# -*- coding: utf-8 -*-

from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class RouterState(TypedDict):
    # 사용자의 질문을 저장합니다.
    question: str

    # 분류 결과를 저장합니다.
    category: str

    # 최종 답변을 저장합니다.
    answer: str

def classify_node(state: RouterState) -> dict:
    """
    질문 내용을 보고 policy 또는 sales 로 분류합니다.
    """
```

```

"""

# 상태에서 질문을 가져옵니다.
question = state["question"]

# 정책 질문으로 판단할 키워드를 정의합니다.
policy_words = ["환불", "배송", "교환", "취소", "적립"]

# 정책 키워드가 하나라도 포함되면 policy 로 분류합니다.
category = (
    "policy"
    if any(word in question for word in policy_words)
    else "sales"
)

# 분류 결과를 상태에 저장합니다.
return {"category": category}

def route_question(state: RouterState) -> str:
    """
    classify 노드 다음에 실행할 노드 이름을 결정합니다.

    반환값:
        policy 또는 sales
    """

    # 앞 노드에서 저장한 category 를 그대로 반환합니다.
    return state["category"]

def policy_node(state: RouterState) -> dict:
    """

```

```

    정책 관련 질문을 처리합니다.
    """

    return {
        "answer": "정책 담당 에이전트가 질문을 처리했습니다."
    }

def sales_node(state: RouterState) -> dict:
    """
    상품 추천 관련 질문을 처리합니다.
    """

    return {
        "answer": "상품 추천 에이전트가 질문을 처리했습니다."
    }

# 상태 구조를 전달하여 그래프를 생성합니다.
graph_builder = StateGraph(RouterState)

# 각 처리 함수를 노드로 등록합니다.
graph_builder.add_node("classify", classify_node)
graph_builder.add_node("policy", policy_node)
graph_builder.add_node("sales", sales_node)

# START 에서 classify 노드로 이동합니다.
graph_builder.add_edge(START, "classify")

# classify 실행 후 route_question 함수의 반환값에 따라 분기합니다.
graph_builder.add_conditional_edges(
    "classify",
    route_question,

```

```

    {
        "policy": "policy",
        "sales": "sales",
    },
)

# 각 전문 노드 실행 후 그래프를 종료합니다.
graph_builder.add_edge("policy", END)
graph_builder.add_edge("sales", END)

# 그래프를 실행 가능한 워크플로우로 컴파일합니다.
workflow = graph_builder.compile()

# 정책 질문을 실행합니다.
result = workflow.invoke(
    {
        "question": "환불은 며칠 이내에 가능한가요?",
        "category": "",
        "answer": "",
    }
)

print(result)

```

7.3 실행 흐름

질문: "환불은 며칠 이내에 가능한가요?"

START

↓

classify

↓

category = "policy"

↓

route_question

↓

policy

↓

END

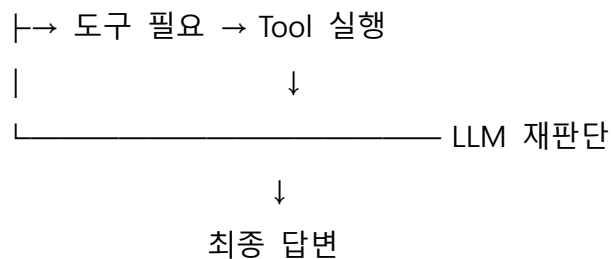
`add_conditional_edges()`는 노드의 상태를 검사하여 다음 실행 노드를 선택합니다. 공식 Graph API 가이드도 조건 분기와 반복 구조를 Graph API 의 핵심 기능으로 설명합니다. ([Docs by LangChain](#))

8. 반복 실행과 Agent Loop

8.1 반복 구조

LangGraph 는 그래프에 순환 Edge 를 추가할 수 있습니다.

LLM 판단



이 구조는 일반적인 ReAct 에이전트의 반복 구조입니다.

Thought → Action → Observation → Thought → Answer

8.2 반복 그래프 예시

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class LoopState(TypedDict):
    # 현재 처리 횟수를 저장합니다.
    count: int
```

```

# 반복 결과를 저장합니다.
message: str

def increase_node(state: LoopState) -> dict:
    """
    count 값을 1 증가시킵니다.
    """

    # 기존 count 값에 1 을 더합니다.
    next_count = state["count"] + 1

    # 증가한 count 와 메시지를 반환합니다.
    return {
        "count": next_count,
        "message": f"{next_count}번째 실행",
    }

def should_continue(state: LoopState) -> str:
    """
    count 값이 3 미만이면 반복하고,
    3 이상이면 그래프를 종료합니다.
    """

    # count 가 3 보다 작으면 다시 increase 로 이동합니다.
    if state["count"] < 3:
        return "repeat"

    # count 가 3 이상이면 END 로 이동합니다.
    return "finish"

```

```

graph_builder = StateGraph(LoopState)

# 반복 실행할 노드를 등록합니다.
graph_builder.add_node("increase", increase_node)

# START 에서 increase 노드를 실행합니다.
graph_builder.add_edge(START, "increase")

# increase 실행 후 조건에 따라 다시 반복하거나 종료합니다.
graph_builder.add_conditional_edges(
    "increase",
    should_continue,
    {
        "repeat": "increase",
        "finish": END,
    },
)

# 실행 가능한 그래프로 컴파일합니다.
workflow = graph_builder.compile()

# count 를 0 부터 시작합니다.
result = workflow.invoke(
    {
        "count": 0,
        "message": "",
    }
)

print(result)

```

실행 흐름은 다음과 같습니다.


```
START
↓
count = 1
↓
count < 3 → 반복
↓
count = 2
↓
count < 3 → 반복
↓
count = 3
↓
count >= 3 → END
```

9. Reducer 와 상태 누적

9.1 기본 상태 업데이트

기본적으로 같은 State 키에 새 값을 반환하면 이전 값이 덮어써집니다.

```
return {"messages": ["새 메시지"]}
```

기존 값:

```
["첫 번째 메시지"]
```

업데이트 후:

```
["새 메시지"]
```

9.2 값을 누적해야 하는 경우

대화 메시지나 검색 결과처럼 기존 값에 새 값을 추가해야 할 수 있습니다.

```
import operator
from typing import Annotated, TypedDict

class MessageState(TypedDict):
    # operator.add Reducer 를 사용하여 기존 리스트와 새 리스트를 합칩니다.
```

```
messages: Annotated[list[str], operator.add]
```

노드가 다음 값을 반환하면:

```
return {"messages": ["두 번째 메시지"]}
```

상태는 다음처럼 누적됩니다.

기존:

```
["첫 번째 메시지"]
```

반환:

```
["두 번째 메시지"]
```

최종:

```
["첫 번째 메시지", "두 번째 메시지"]
```

병렬 노드가 같은 State 키를 동시에 갱신할 경우 Reducer 가 없으면 어떤 값을 적용해야 할지 결정할 수 없어 오류가 발생할 수 있습니다. 공식 오류 문서도 병렬 실행 시 같은 상태 키를 여러 노드가 갱신한다면 Reducer 를 정의해야 한다고 설명합니다. ([Docs by LangChain](#))

10. MessagesState

대화형 에이전트에서는 직접 State 를 정의하지 않고 MessagesState 를 사용할 수 있습니다.

```
from langgraph.graph import MessagesState
# MessagesState 는 메시지 목록을 관리하기 위한 기본 상태 구조입니다.
from langgraph.graph import StateGraph, MessagesState, START, END

def chatbot_node(state: MessagesState) -> dict:
    """
    가장 최근 사용자 메시지를 확인하고 AI 메시지를 반환합니다.
    """

    # 메시지 목록에서 마지막 메시지를 가져옵니다.
    last_message = state["messages"][-1]
```

```

# 간단한 AI 응답을 생성합니다.
answer = f"입력한 내용은 '{last_message.content}'입니다."

# messages 에 추가할 AI 메시지를 반환합니다.
return {
    "messages": [
        {
            "role": "assistant",
            "content": answer,
        }
    ]
}

# MessagesState 를 사용하는 그래프를 생성합니다.
graph_builder = StateGraph(MessagesState)

# chatbot 노드를 등록합니다.
graph_builder.add_node("chatbot", chatbot_node)

# 실행 순서를 연결합니다.
graph_builder.add_edge(START, "chatbot")
graph_builder.add_edge("chatbot", END)

# 그래프를 컴파일합니다.
workflow = graph_builder.compile()

# 사용자 메시지를 입력하여 그래프를 실행합니다.
result = workflow.invoke(
    {
        "messages": [
            {

```

```

        "role": "user",
        "content": "LangGraph 가 무엇인가요?",
    }
]
}
)

print(result["messages"][-1].content)

```

공식 Overview 에서도 MessagesState 를 사용한 기본 그래프 예제를 제공합니다. ([Docs by LangChain](#))

11. LangGraph 기반 멀티에이전트

멀티에이전트는 여러 역할의 에이전트를 그래프에 연결한 구조입니다.

사용자 질문

↓

Supervisor

└→ Sales Agent

└→ Policy Agent

└→ Order Agent

└→ Research Agent

State 예시는 다음과 같습니다.

```

class MultiAgentState(TypedDict):
    # 사용자가 입력한 질문입니다.
    question: str

    # Supervisor 가 선택한 담당 에이전트 이름입니다.
    target_agent: str

    # 전문 에이전트가 생성한 결과입니다.

```

```

agent_result: str

# 최종 응답입니다.
final_answer: str
라우팅 함수:
def route_agent(state: MultiAgentState) -> str:
    """
    Supervisor 가 결정한 에이전트 이름을 다음 노드 이름으로 반환합니다.
    """

    return state["target_agent"]
그래프 연결:
graph_builder.add_conditional_edges(
    "supervisor",
    route_agent,
    {
        "sales": "sales_agent",
        "policy": "policy_agent",
        "order": "order_agent",
    },
)

```

12. 실무 설계 원칙

12.1 State 는 필요한 정보만 저장

좋지 않은 구조:

```

class State(TypedDict):
    everything: dict

```

권장 구조:

```
class State(TypedDict):  
    question: str  
    category: str  
    documents: list[str]  
    answer: str  
    retry_count: int  
    error: str
```

12.2 Node 는 하나의 책임만 수행

좋지 않은 Node:

분류 + 검색 + 답변 + DB 저장 + 로그 저장

권장 Node:

classify_node
retrieve_node
grade_node
generate_node
save_node

12.3 LLM 이 필요 없는 일에는 Python 사용

LLM 사용:

- 자연어 분류
- 요약
- 답변 생성
- 의미 평가

Python 사용:

- 숫자 계산
- DB 저장
- 문자열 검증

- 권한 검사
- 카테고리 매핑
- 반복 횟수 검사

12.4 LLM 출력은 반드시 검증

```
ALLOWED_ROUTES = {"sales", "policy", "order"}

def validate_route(raw_route: str) -> str:
    """
    LLM의 라우팅 결과가 허용 범위 안인지 검사합니다.
    """

    # 앞뒤 공백을 제거하고 소문자로 통일합니다.
    normalized = raw_route.strip().lower()

    # 허용된 값이면 그대로 사용합니다.
    if normalized in ALLOWED_ROUTES:
        return normalized

    # 허용 범위를 벗어나면 안전한 기본 경로로 보정합니다.
    return "policy"
```

12.5 모든 반복에는 종료 조건 설정

반복 조건 + 최대 횟수 + fallback

12.6 운영에서는 Checkpointer 사용

개발:

InMemorySaver

로컬:

SQLite

운영:

PostgreSQL 또는 영구 저장소

12.7 Node 별 로그 기록

```
def classify_node(state: dict) -> dict:
    logger.info(
        "classify_node 시작: question=%s",
        state["question"],
    )

    # 분류 처리
    category = "policy"

    logger.info(
        "classify_node 완료: category=%s",
        category,
    )

    return {"category": category}
```

13. LangGraph 가 적합한 경우

다음 조건 중 여러 개가 해당하면 LangGraph 사용을 고려할 수 있습니다.

- 실행 과정에 분기가 있다.
- 같은 단계를 반복할 수 있다.
- 여러 처리 단계가 State를 공유한다.
- 도구 호출 결과에 따라 다음 행동이 달라진다.
- 작업을 중단했다가 다시 시작해야 한다.
- 사람의 승인이 필요하다.
- 멀티에이전트를 구현한다.
- 긴 작업의 진행 상태를 스트리밍한다.
- 실패 지점부터 복구해야 한다.

14. LangGraph 가 필요하지 않은 경우

다음과 같은 단순한 작업에는 LangGraph 가 지나치게 복잡할 수 있습니다.

- LLM 을 한 번 호출하고 답변만 받음
- 단순 번역
- 단순 요약
- 고정된 두 단계 처리
- 분기나 반복이 전혀 없음

예:

```
response = llm.invoke("다음 문장을 영어로 번역하세요.")
```

이 정도 작업에는 일반 LangChain 또는 LLM SDK 만으로 충분합니다.

15. 핵심 API 정리

API	설명
StateGraph(State)	상태 기반 그래프 생성
add_node(name, function)	처리 노드 등록
add_edge(source, target)	고정 실행 경로 연결
add_conditional_edges()	상태에 따른 조건부 분기
compile()	실행 가능한 그래프로 변환
invoke()	동기 방식으로 한 번 실행
ainvoke()	비동기 방식으로 실행
stream()	중간 실행 결과 스트리밍
get_graph()	그래프 구조 조회
START	그래프 시작점
END	그래프 종료점
MessagesState	대화 메시지용 기본 State
interrupt()	실행 중단 및 외부 입력 대기
Command	상태 갱신과 이동 제어
Send	동적 병렬 작업 전달

3. Multi agent concept

만능 한 명 vs 전문가 팀

이전까지는 도구를 여러 개 단 **하나의 에이전트**를 만들었습니다. 기능이 쌓이면 프롬프트가 수백 줄이 되고 도구가 10개를 넘어 LLM이 헛갈리죠. 해결책은 **역할별 전문 에이전트**로 쪼개고, 앞단에 라우터(Supervisor)를 두는 멀티에이전트입니다.

1. 만능 한 명의 한계

승승장구물 CS 에이전트에 기능이 하나씩 쌓이는 상황을 상상해 봅시다.

처음: 주문 조회만 → 잘 됨

한 달 뒤: 주문 조회 + 환불 정책 + 상품 추천 + 쿠폰 + 재고 + 배송조회...

→ 시스템 프롬프트가 "주문은 이렇게, 환불은 저렇게, 추천은 또..." 수백 줄

→ 도구가 get_order, get_stock, recommend, policy_search, refund, coupon... 10개+

→ LLM이 "어느 도구를 써야 하지?" 자주 헛갈림

→ 한 군데 고치면 다른 기능이 깨짐

한 명에게 모든 일을 시키니, 만능이 아니라 어중간해지는 거죠.

2. 백엔드 개발자에게 익숙한 문제

이건 사실 백엔드 개발자가 잘 아는 문제입니다.

[하나의 거대한 God Object 서비스]

모든 기능이 한 클래스/서비스에 → 비대 · 결합 · 수정 위험

↓ 해결책

[마이크로서비스 + API Gateway]

책임별로 서비스를 쪼개고, 앞단 라우터가 알맞은 곳으로 보냄

에이전트 세계의 해결책도 똑같습니다. 역할(책임)별로 에이전트를 쪼개고, 앞단에 라우터를 둡니다. 이게 멀티에이전트(Multi-Agent)입니다.

3. 전문가 팀으로 나누기

승승장구를 CS를 전문가 팀으로 재구성하면:

- [만능 한 명] 하나의 에이전트 — 프롬프트 수백 줄 / 도구 10개+ / 도구 선택 혼동
- [전문가 팀] **Supervisor(라우터)** 가 질문을 알맞은 전문가에게 보냄:
- 추천 전문 에이전트 (상품 도구만)
- 정책 전문 에이전트 (FAQ 도구만)
- 주문 전문 에이전트 (조회 도구만)

각 전문가는:

- 짧은 프롬프트 (자기 역할만)
- 적은 도구 (자기 책임만) — 도구 선택 정확
- 독립적 (한 명 고쳐도 다른 명 안 깨짐)

회사에서도 "한 명이 모든 부서 일"보다 "부서별 전문가 + 안내데스크"가 효율적이듯, 에이전트도 마찬가지죠.

4. 멀티에이전트의 진짜 의미

흔한 오해부터 바로잡습니다.

- [오해] 멀티에이전트 = AI를 여러 개 띄우는 것
- [진짜] 멀티에이전트 = 역할(책임)별로 프롬프트와 도구 집합을 분리하고,
그 사이의 흐름(라우팅/인계)을 코드로 설계하는 일

핵심은 "AI 개수"가 아니라 "책임 분리 + 흐름 설계" 입니다. 같은 LLM을 쓰더라도, 프롬프트와 도구를 역할별로 나누고 라우터로 연결하면 멀티에이전트죠. 24강까지의 "단일 에이전트 + 여러 도구"를, "여러 전문 에이전트 + 라우터"로 재구성하는 겁니다.

5. 우리가 만들 것

이번 강에서는 두 전문 에이전트 + **Supervisor**를 만듭니다.

고객 질문 → [**Supervisor 라우터**] "이건 추천? 정책?"

- 추천 질문 → [**추천 에이전트**] (products.csv 도구)
- 정책 질문 → [**정책 에이전트**] (faq.csv 도구)

→ 알맞은 전문가의 답변

- 추천 에이전트(sales): `products.csv` 로 상품 추천.
- 정책 에이전트(policy): `faq.csv` 로 환불·배송 등 정책 안내.
- **Supervisor**: 질문을 보고 둘 중 하나에게 위임.

그리고 라우터를 규칙(키워드) vs LLM(의미 이해) 두 방식으로 만들어 비교합니다.

6. 언제 쪼개야 하나

"무조건 쪼개라"는 아닙니다. 분리 시점의 기준:

- 도구가 5~7개를 넘는다
- 책임이 명확히 갈린다 (추천 vs 정책 vs 주문)
- 프롬프트가 너무 길어 LLM이 혼동한다
- 한 기능 수정이 다른 기능을 깨뜨린다

반대로 도구 2~3개의 작은 에이전트는 굳이 쪼갤 필요 없습니다. 쪼개면 라우팅 복잡도가 늘어나니까요. "God Object가 됐을 때 쪼갬다" — 이 판단력이 중요합니다.

핵심 정리

- 만능 한 에이전트는 기능이 쌓이면 **프롬프트 비대·도구 과다·수정 위험**.
- 이건 백엔드의 **God Object** 문제 → 해결책은 **마이크로서비스 + 라우터**.
- 멀티에이전트 = **역할별 프롬프트·도구 분리 + 흐름(라우팅) 설계**(AI 개수 아님).
- 각 전문 에이전트는 **짧은 프롬프트·적은 도구**로 정확·독립적.
- 이번 강: **추천/정책 에이전트 2개 + Supervisor 라우터**.
- 쪼개는 시점: 도구 5~7개 초과·책임 분리·혼동·수정 위험.

단일 에이전트의 한계

단일(거대) 에이전트와 멀티에이전트를 항목별로 비교하면 차이가 분명합니다. 도구가 5~7개를 넘고 책임이 명확히 갈린다면, 역할별로 쪼개 도구 선택 정확도·유지보수·비용을 모두 개선할 시점입니다.

1. 항목별 비교

항목	단일(거대) 에이전트	멀티에이전트
시스템 프롬프트	모든 규칙을 한 곳에(수백 줄)	역할별로 짧게 분리
도구 수	전부 한 에이전트에(과다)	역할에 필요한 것만
도구 선택 정확도	낮음(혼동)	높음(후보가 적음)
유지보수	수정 시 부작용 큼	역할 단위로 격리
테스트	통합 테스트만	에이전트별 단위 테스트
비용/지연	긴 프롬프트로 토큰 ↑	짧은 프롬프트로 절약

하나씩 자세히 봅시다.

2. 도구 선택 정확도 — 핵심 문제

가장 중요한 차이입니다. 6강에서 배웠듯, LLM은 도구 설명(docstring)을 읽고 어느 도구를 쓸지 판단합니다. 도구가 많아지면 이 판단이 흔들리죠.

[도구 10개인 단일 에이전트]

"환불 문의" → `get_order?` `refund?` `policy_search?` `coupon?`
→ 비슷한 도구들 사이에서 LLM이 헛갈림 → 엉뚱한 도구 호출

[도구 1개인 정책 전문 에이전트]

"환불 문의" → `search_faq` (선택지가 하나라 헛갈릴 수 없음)
→ 정확

후보가 적으면 선택이 정확합니다. 사람도 "10개 메뉴 중 고르기"보다 "2개 중 고르기"가 쉽듯, LLM도 도구가 적을수록 정확하죠. 역할 분리의 가장 직접적인 효과입니다.

3. 프롬프트 비대화

단일 에이전트는 모든 규칙을 한 프롬프트에 옥여넣습니다.

```
# 단일 에이전트의 프롬프트 (계속 길어짐)
system_prompt = """
너는 승승장구몰 상담원이다.
- 주문 조회는 주문번호를 받아 get_order로...
- 환불은 7일 이내만 가능하고 절차는...
- 상품 추천은 카테고리를 받아...
- 쿠폰은 등급별로 다르고...
- 배송 조회는...
(수백 줄 계속)
"""
```

```
# 멀티에이전트 — 각자 짧게
sales_prompt = "너는 '상품 추천' 전문 상담원이다. 추천 외 질문은 정책 담당에 안내하라."
policy_prompt = "너는 '정책/FAQ 안내' 전문 상담원이다. FAQ 근거로만 답하라."
```

프롬프트가 짧아지면:

- LLM이 자기 역할에 집중 (잡음 적음)
- 토큰이 줄어 비용 · 지연 감소 (매 호출마다 긴 프롬프트를 안 보냄)
- 사람이 읽고 수정하기 쉬움

4. 유지보수 — 격리의 가치

단일 에이전트는 한 곳을 고치면 다른 곳이 깨질 위험이 있습니다.

```
[단일] "환불 정책 문구 수정" → 같은 프롬프트의 추천 로직에 영향?
→ 전체를 다시 테스트해야 안심

[멀티] "환불 정책 수정" → policy_agent만 수정 · 테스트
→ sales_agent는 건드리지 않았으니 안전
```

역할별로 **격리**돼 있어, 수정 범위가 명확합니다. 백엔드의 마이크로서비스가 "한 서비스 배포가 다른 서비스에 영향 없게" 하는 것과 같죠.

5. 테스트 — 단위 테스트 가능

[단일] 통합 테스트만 가능 — "전체 시나리오를 돌려봐야" 검증
[멀티] 에이전트별 단위 테스트 — "추천 에이전트만" 따로 검증 가능

```
# 멀티에이전트는 각 에이전트를 독립적으로 테스트
def test_sales_agent():
    r = sales_agent.invoke({"messages": [{"role": "user", "content": "전자기기 추천"}]})
    assert "원" in r["messages"][-1].content # 상품 추천이 나왔는지

def test_policy_agent():
    r = policy_agent.invoke({"messages": [{"role": "user", "content": "환불 기간"}]})
    assert "7일" in r["messages"][-1].content # 정책 답변이 나왔는지
```

작은 단위로 테스트하니 어디가 문제인지 빨리 찾습니다.

6. 분리의 비용 — 공짜가 아니다

물론 멀티에이전트도 단점이 있습니다.

- 라우팅 로직이 추가됨 (Supervisor를 만들고 관리)
- 라우터가 틀리면 엉뚱한 에이전트로 감 (오라우팅)
- 에이전트 간 흐름 추적이 복잡해질 수 있음
- 너무 잘게 쪼개면 오히려 관리 부담

그래서 "무조건 쪼개라"가 아니라 "God Object가 됐을 때 쪼개라" 입니다. 도구 2~3개의 작은 에이전트는 단일로 두는 게 낫죠. 트레이드오프를 보고 판단하는 게 핵심입니다

핵심 정리

- 단일 vs 멀티: 프롬프트·도구·정확도·유지보수·테스트·비용 전반에서 차이.
- 핵심 효과: 도구 후보가 적어 도구 선택 정확도 ↑ (6강 docstring 매칭).
- 프롬프트가 짧아져 집중·비용·가독성 개선.
- 역할별 격리로 수정 범위 명확, 단위 테스트 가능.
- 단, 라우팅 복잡도·오라우팅 위험이라는 비용도 존재.
- 판단 기준: 도구 5~7개 초과·책임 분리·혼동 → 그때 쪼갬다.

Supervisor vs Handoff

멀티에이전트의 협업 구조는 크게 둘입니다. Supervisor(감독자) 는 가운데 라우터가 질문을 알맞은 전문가에게 보내는 방식(API Gateway), Handoff(인계) 는 에이전트끼리 직접 넘기는 방식입니다. 이번 절은 가장 흔하고 이해하기 쉬운 Supervisor 라우팅을 구현합니다.

1. Supervisor(감독자) 패턴 — 가운데 라우터

사용자 질문 → **[Supervisor]** "이건 누구 일?"(라우터)이 알맞은 전문가에게 위임합니다.

- → **[추천 에이전트]** (+추천 도구)
- → **[정책 에이전트]** (+RAG 도구)
- → **[주문 에이전트]** (+조회 도구)

→ 위임받은 전문가가 최종 답변.

- Supervisor가 질문을 보고 어느 전문 에이전트에게 보낼지 결정합니다.
- 백엔드의 **API Gateway** / 리버스 프록시와 똑같은 역할이죠.
- 라우터는 (a) 규칙(키워드) 또는 (b) LLM 분류로 구현합니다(05-06번).

장점: 흐름이 중앙에서 통제돼 추적·관리가 쉽습니다. "이 질문이 어디로 갔는지" Supervisor 한 곳만 보면 알죠.

2. Handoff(인계) 패턴 — 에이전트끼리 넘김

1. 사용자 질문 → **[추천 에이전트]**: "어, 이건 환불 질문이네. 정책팀에 넘길게"
2. 인계 → **[정책 에이전트]** 가 답변

- 한 에이전트가 자기 일이 아니라고 판단하면 다른 에이전트에게 직접 인계합니다.
- 중앙 라우터 없이 동작합니다.
- 26·27강의 순차 파이프라인도 일종의 "정해진 handoff"입니다.

단점: 중앙 통제가 없어 흐름 추적이 어려울 수 있습니다. "A가 B에게, B가 다시 C에게..." 넘기다 보면 어디서 무슨 일이 있었는지 따라가기 복잡해지죠.

3. 둘의 비교

	Supervisor	Handoff
구조	중앙 라우터가 위임	에이전트끼리 직접 인계
비유	API Gateway	서비스 간 호출
흐름 추적	쉬움(중앙 집중)	어려움(분산)
유연성	라우터가 병목/단일 통제	자유롭지만 통제 어려움
적합	명확한 역할 구분	동적·연쇄적 협업

운영에서는 **Supervisor**가 흔합니다. 흐름이 명확하고 디버깅이 쉬우니까요. Handoff는 더 자유롭지만, 통제가 어려워 신중히 써야 합니다.

4. 에이전트 간 통신 방식 4가지

협업 구조를 더 넓게 보면, 에이전트들이 소통하는 방식은 네 가지입니다.

방식	설명	비유(백엔드)
순차 파이프라인	A→B→C 정해진 순서로 결과 전달	배치 파이프라인
Supervisor 라우팅	라우터가 1명을 골라 위임	API Gateway
Handoff	에이전트가 서로 넘김	서비스 간 호출
상태 공유 그래프	공유 State로 협업	공유 DB/큐

- 순차 파이프라인 → 26강(Planner/Worker), 27강(AI팀)
- Supervisor 라우팅 → 이번 25강
- 상태 공유 그래프 → 27강 (LangGraph StateGraph)

5. 무엇을 주고받는가

협업의 핵심은 에이전트 사이에 무엇을 전달하느냐입니다. 보통 세 가지를 주고받죠.

1. 사용자 질문 — 원래 요청 그대로
2. 직전 에이전트의 산출물 — 파이프라인에서 앞 단계 결과
3. 공유 상태(State) — 모두가 함께 읽고 쓰는 데이터 (27강)

출발점인 Supervisor는 주로 (1) **사용자 질문**을 전달합니다. Supervisor가 질문을 받아, 그대로 알맞은 에이전트에게 넘기죠. 27강에서는 (3) **공유 State**를 LangGraph StateGraph로 정식화해, 여러 에이전트가 하나의 상태를 함께 채우는 협업을 만듭니다.

6. 왜 Supervisor부터 배우나

이번 강이 Supervisor를 먼저 다루는 이유:

- 가장 직관적 (안내데스크 = 라우터, 이해 쉬움)
- 운영에서 가장 흔함 (대부분의 멀티에이전트가 이 형태)
- 흐름 추적이 쉬워 디버깅 친화적
- 라우팅 개념(규칙 vs LLM)을 깔끔히 배울 수 있음

Supervisor의 핵심은 라우팅입니다 — "이 질문을 누구에게 보낼까?"를 결정하는 것. 04번에서 전문 에이전트 2개를 만들고, 05번에서 이들을 잇는 Supervisor 라우터를 구현합니다. 그리고 06번에서 규칙 라우터와 LLM 라우터를 비교하죠.

핵심 정리

- **Supervisor**: 중앙 라우터가 질문을 알맞은 전문가에게 위임(API Gateway).
- **Handoff**: 에이전트끼리 직접 인계(중앙 통제 없음, 추적 어려움).
- 통신 방식 4가지: 순차 파이프라인·Supervisor·Handoff·상태 공유 그래프.
- 전달 대상: 사용자 질문·직전 산출물·공유 State(27강).
- 운영에서는 **추적 쉬운 Supervisor가 흔함** → 이번 강의 주제.

따라하기 — 역할별 에이전트 2개

목표

추천 전문(sales)과 정책 전문(policy), 서로 다른 프롬프트·도구를 가진 두 에이전트를 만듭니다. 각자 자기 책임의 도구만 보게 해 도구 선택 정확도를 높입니다.

0. 실습 준비

```
conda activate agentic
pip install -U langgraph langchain langchain-google-genai pandas
python code/common.py          # GOOGLE_API_KEY: True 확인
```

- langgraph·langchain: create_agent 사용.
- pandas: CSV 도구.
- 데이터: data/products.csv(상품), data/faq.csv(정책 FAQ).

1. 공통 시작부와 데이터 로드

`code/ch25_multi_agent.py` 를 만듭니다.

```
# -*- coding: utf-8 -*-
# 실행: python code/ch25_multi_agent.py
import sys, pathlib
sys.path.append(str(pathlib.Path(__file__).resolve().parent)) # code/ 를 import 경로에
from common import get_chat, DATA

import pandas as pd
from langchain_core.tools import tool          # 함수를 LLM이 쓸 수 있는 '도구'로 등록
from langchain.agents import create_agent

# [왜 멀티에이전트] 한 에이전트에 도구가 너무 많아지면 LLM이 '어느 도구를 언제 쓸지'
# 헛갈린다. 그래서 역할(추천/정책)별로 에이전트를 쪼개고(역할 분리), 질문이 들어오면
# supervisor 가 알맞은 에이전트로 보낸다(라우팅). = '역할 분리 + 라우팅' 패턴.

products = pd.read_csv(DATA / "products.csv") # product_id,product_name,category,price,cost,stock,rating
faq = pd.read_csv(DATA / "faq.csv")          # question,answer
```

2. 역할별 도구 — 책임에 맞는 도구만

각 에이전트가 쓸 도구를 `@tool`로 정의합니다(6강). 추천용과 정책용을 따로 만듭니다.

```
@tool
def search_products(category: str) -> str:
    """카테고리명을 받아 승승장구몰에서 평점 높은 상품 3개를 추천한다.

    @tool 데코레이터: 이 docstring 이 LLM에게 '도구 설명서'로 전달된다.
    LLM은 설명을 읽고 이 도구를 언제 호출할지 스스로 판단한다.
    """
    hit = products[products["category"].str.contains(category, na=False)]
    if hit.empty:
        return f"'{category}' 카테고리 상품을 찾지 못했습니다."
    top = hit.sort_values("rating", ascending=False).head(3)
    return "\n".join(
        f"- {r.product_name} ({r.price:,}원, 평점 {r.rating})"
        for r in top.itertuples()
    )

@tool
def search_faq(keyword: str) -> str:
    """키워드로 승승장구몰 FAQ에서 가장 관련 있는 답변을 찾는다."""
    hit = faq[faq["question"].str.contains(keyword, na=False)]
    if hit.empty:
        # 질문에 없으면 답변 본문에서도 검색
        hit = faq[faq["answer"].str.contains(keyword, na=False)]
    if hit.empty:
        return "관련 FAQ를 찾지 못했습니다."
    return "\n".join(f"Q. {r.question}\nA. {r.answer}" for r in hit.head(2).itertuples())
```

도구 설계 포인트

- `search_products`: 카테고리로 상품 검색, 평점순 3개. **추천 에이전트 전용.**
- `search_faq`: 키워드로 FAQ 검색, 질문→답변 본문 순으로 fallback. **정책 에이전트 전용.**
- docstring이 **LLM에게 전달되는 도구 설명서**입니다. 명확하게 써야 LLM이 제대로 호출하죠.

3. 역할별 에이전트 2개 생성

핵심입니다. 두 에이전트를 다른 프롬프트 + 다른 도구로 만듭니다.

```
def build_agents():
    """역할별 에이전트 2개(추천/정책)를 생성해 반환한다(프롬프트 · 도구 분리).

    [왜 도구 분리] 추천 에이전트엔 search_products 만, 정책 에이전트엔 search_faq 만
    준다. 각 에이전트가 자기 책임의 도구만 보면 헷갈릴 일이 줄어 정확해진다.
    """
    llm = get_chat(temperature=0) # [왜 0] 도구 선택 · 분류는 정해진 답이라 무작위성 제거

    sales_agent = create_agent( # 추천 전문 에이전트
        llm,
        tools=[search_products], # 추천 도구만!
        system_prompt="너는 승승장구몰의 '상품 추천' 전문 상담원이다. "
            "상품 추천 외 질문은 답하지 말고 '정책 담당에게 문의하라'고 안내하라.",
    )
    policy_agent = create_agent( # 정책/FAQ 전문 에이전트
        llm,
        tools=[search_faq], # 정책 도구만!
        system_prompt="너는 승승장구몰의 '정책/FAQ 안내' 전문 상담원이다. "
            "환불 · 배송 · 교환 · 적립 등 정책만 답하고, 반드시 FAQ 근거로만 답하라.",
    )
    return llm, sales_agent, policy_agent
```

4. 무엇이 분리됐나

두 에이전트의 차이를 표로 보면:

	sales_agent	policy_agent
도구	<code>search_products</code> 만	<code>search_faq</code> 만
프롬프트	"상품 추천 전문"	"정책/FAQ 전문"
책임	상품 추천	환불·배송·교환·적립
범위 밖 질문	"정책 담당에 문의" 안내	정책만 답

프롬프트도 다르고 도구도 다릅니다. 이게 역할 분리의 핵심이죠. 각 에이전트가:

- 짧은 프롬프트로 자기 역할에 집중
- 도구가 1개뿐이라 도구 선택 혼동 없음
- 범위 밖 질문은 "다른 담당에 문의"로 선 긋기

특히 sales_agent의 "추천 외 질문은 정책 담당에 문의하라" 가 중요합니다. 자기 책임을 명확히 알고, 아니면 떠넘기지 않고 안내하죠.

5. 에이전트 단독 테스트

Supervisor를 붙이기 전에, 각 에이전트가 잘 동작하는지 따로 확인합니다.

```
if __name__ == "__main__":
    llm, sales_agent, policy_agent = build_agents()

    # 추천 에이전트 단독 테스트
    r = sales_agent.invoke({"messages": [{"role": "user", "content": "전자기기 추천해줘"}]})
    print("[추천 에이전트]", r["messages"][-1].content[:120])

    # 정책 에이전트 단독 테스트
    r = policy_agent.invoke({"messages": [{"role": "user", "content": "환불 며칠 이내?"}]}))
    print("[정책 에이전트]", r["messages"][-1].content[:120])
```

예상 출력

```
[추천 에이전트] 전자기기 카테고리 추천 상품입니다.
- 승승 스마트워치 Fit 5 (159,000원, 평점 4.8)
- ...
[정책 에이전트] 교환/환불은 상품 수령 후 7일 이내 신청 가능합니다. ...
```

각 에이전트가 자기 도구를 써서 자기 영역의 답을 합니다. 02번에서 말한 "에이전트별 단위 테스트"가 바로 이거죠. 이렇게 따로 검증한 뒤, 05번에서 Supervisor로 묶습니다.

핵심 정리

- @tool로 추천용(search_products)-정책용(search_faq) 도구를 따로 정의.
- create_agent로 프롬프트-도구가 다른 두 에이전트 생성.
- sales_agent엔 추천 도구만, policy_agent엔 정책 도구만 → 도구 선택 정확.
- 각 프롬프트에 역할·범위 밖 처리("다른 담당에 문의")를 명시.
- temperature=0 — 도구 선택은 정해진 답이라 무작위성 제거.
- Supervisor 전에 에이전트별 단독 테스트로 검증.

따라하기 — Supervisor 라우터

목표

04번의 두 전문 에이전트를 잇는 **Supervisor** 라우터를 만듭니다. 질문을 보고 추천/정책 중 알맞은 에이전트에게 위임하죠. 라우터를 규칙(키워드) 과 LLM(의미 이해) 두 방식으로 구현합니다.

1. Supervisor의 역할

Supervisor는 "이 질문을 누구에게 보낼까?" 를 결정하는 라우터입니다.

질문 → [Supervisor: 분류] → "policy" 또는 "sales" → 해당 에이전트에 위임 → 답변

분류 방법은 두 가지입니다.

- (A) 규칙 라우터 — 키워드 매칭 (빠름 · 공짜)
- (B) LLM 라우터 — 의미 이해 (정확하지만 비용)

먼저 규칙으로 만들고, 규칙이 못 잡는 애매한 질문을 LLM으로 보완합니다.

2. (A) 규칙 기반 라우터 — 빠르고 공짜

정책 관련 키워드 목록을 만들고, 질문에 그 단어가 있으면 정책으로 보냅니다.

```
# supervisor: 여러 에이전트 중 '어디로 보낼지'를 결정하는 라우팅 담당.
POLICY_WORDS = ["환불", "교환", "배송", "취소", "적립", "포인트", "무료배송", "회원", "등급"]

def route_rule(question: str) -> str:
    """질문을 'policy' 또는 'sales' 로 분류한다(규칙 기반, 비용 0).

    [왜 규칙 라우터] 키워드 매칭은 LLM 호출이 없어 빠르고 공짜다.
    명확한 정책 단어가 있으면 굳이 LLM에 물어볼 필요가 없다.
    """
    return "policy" if any(w in question for w in POLICY_WORDS) else "sales"
```

작동 방식

"환불은 며칠 안에?" → '환불' 포함 → "policy"
 "전자기기 추천해줘" → 정책 단어 없음 → "sales" (기본값)

`any(w in question for w in POLICY_WORDS)` — 정책 단어 중 하나라도 있으면 True. 없으면 기본값 "sales"로 보냅니다. LLM 호출이 없어 0초·0원이죠. 명확한 키워드는 이걸로 충분합니다.

3. (B) LLM 기반 라우터 — 애매한 질문까지

키워드로 안 잡히는 질문은 LLM의 의미 이해로 분류합니다.

```
def route_llm(llm, question: str) -> str:
    """LLM에게 분류만 시킨다. 'policy' 또는 'sales' 한 단어만 받는다.

    [왜 LLM 라우터] '가성비 좋은 거'처럼 키워드로 안 잡히는 애매한 질문은
    LLM의 의미 이해로 분류한다(비용은 들지만 정확). 규칙→LLM 2단계 전략.
    """
    msg = (
        "다음 고객 질문을 한 단어로 분류하라. "
        "환불/배송/교환/적립 등 정책이면 'policy', 상품 추천이면 'sales'.\n"
        f"질문: {question}\n분류(policy/sales):"
    )
    ans = llm.invoke(msg).content.strip().lower()
    return "policy" if "policy" in ans else "sales"
```


작동 방식

"주문한 거 무를 수 있어?" → 키워드 '취소' 없지만 LLM은 의미로 이해 → "policy"
"가성비 좋은 거 없을까?" → LLM이 추천 질문으로 이해 → "sales"

LLM이 분류만 하게 한 단어("policy"/"sales")로 답하도록 프롬프트를 좁혔습니다. "policy" in ans로 결과를 파싱하죠. 호출 1회 비용이 들지만, 규칙이 못 잡는 애매한 질문을 정확히 분류합니다.

4. Supervisor — 분류 후 위임

두 라우터를 골라 쓰고, 분류 결과로 알맞은 에이전트에게 위임합니다.

```
def supervisor(llm, sales_agent, policy_agent, question: str, use_llm: bool = False) -> str:
    """질문을 분류해 적절한 전문 에이전트에 위임하고 답을 돌려준다(라우팅의 핵심)."""
    target = route_llm(llm, question) if use_llm else route_rule(question) # 라우터 선택
    agent = policy_agent if target == "policy" else sales_agent # 위임 대상 결정
    print(f" [Supervisor] '{question}' → {target} 에이전트로 라우팅")
    result = agent.invoke({"messages": [{"role": "user", "content": question}]})
    return result["messages"][-1].content # 위임받은 에이전트의 최종 답변
```

흐름

1. use_llm 플래그로 규칙/LLM 라우터 선택
2. 라우터가 "policy" 또는 "sales" 반환
3. 분류 결과로 위임 대상(policy_agent/sales_agent) 결정
4. 그 에이전트에게 질문을 넘겨 실행
5. 에이전트의 최종 답변 반환

use_llm=False (기본)면 규칙 라우터, True면 LLM 라우터를 씁니다. 라우팅 로그([Supervisor] ... → ...)를 찍어 어디로 갔는지 보이게 했죠.

5. 실행

```
def main():
    llm, sales_agent, policy_agent = build_agents()
    questions = [
        "전자기기 카테고리에서 뭐 살만한 거 추천해줘",
        "환불은 며칠 안에 신청해야 해?",
        "무료배송 기준이 어떻게 돼?",
    ]
    for q in questions:
        print("=" * 60)
        print("고객:", q)
        print("답변:", supervisor(llm, sales_agent, policy_agent, q))    # 규칙 라우터
    print("=" * 60)
    print("고객: 가성비 좋은 거 없을까?")
    print("답변:", supervisor(llm, sales_agent, policy_agent,
                              "가성비 좋은 거 없을까?", use_llm=True))    # LLM 라우터

if __name__ == "__main__":
    main()
```

예상 출력

```
=====
고객: 전자기기 카테고리에서 뭐 살만한 거 추천해줘
[Supervisor] '전자기기 카테고리에서 뭐 살만한 거 추천해줘' → sales 에이전트로 라우팅
답변: 전자기기 카테고리 추천 상품입니다.
- 승승 스마트워치 Fit 5 (159,000원, 평점 4.8) ...

=====
고객: 환불은 며칠 안에 신청해야 해?
[Supervisor] '환불은 며칠 안에 신청해야 해?' → policy 에이전트로 라우팅
답변: 교환/환불은 상품 수령 후 7일 이내 신청 가능합니다. ...

=====
고객: 가성비 좋은 거 없을까?
[Supervisor] '가성비 좋은 거 없을까?' → sales 에이전트로 라우팅
답변: 가성비 좋은 추천 상품입니다. ...
```

관찰 포인트

- 같은 질문 풀인데 서로 다른 전문 에이전트가 처리합니다.
- 규칙 라우터는 '환불'/'무료배송' 키워드로 정확히 정책으로 보냈죠.
- "가성비 좋은 거"는 키워드가 없어 LLM 라우터(`use_llm=True`)로 분류했습니다.

6. 하이브리드 전략 — 규칙 먼저, 안 잡히면 LLM

실무에서는 두 라우터를 **2단계로 결합**하는 게 흔합니다.

```
def route_hybrid(llm, question: str) -> str:
    """규칙으로 1차 거르고, 애매하면 LLM으로 2차 분류 (정확도 · 비용 둘 다 잡기)."""
    # 명확한 정책 키워드가 있으면 LLM 호출 없이 즉시 결정 (공짜)
    if any(w in question for w in POLICY_WORDS):
        return "policy"
    # 키워드가 없으면 애매할 수 있으니 LLM에게 (비용 발생, 정확)
    return route_llm(llm, question)
```

"환불 기간?" → 규칙이 즉시 policy (LLM 호출 0)
"주문 무를 수 있어?" → 규칙 못 잡음 → LLM이 policy (LLM 호출 1)

명확한 질문은 공짜로 거르고, 애매한 질문만 LLM에 물어 비용과 정확도를 동시에 잡습니다. 06번에서 두 라우터의 정확도·비용을 실제로 측정해 이 전략의 근거를 봅니다.

핵심 정리

- **Supervisor**: 질문을 분류해 알맞은 전문 에이전트에 위임하는 라우터.
- **규칙 라우터(route_rule)**: 키워드 매칭, **빠름·공짜**, 명확한 질문에 충분.
- **LLM 라우터(route_llm)**: 의미 이해, **정확하지만 비용**, 애매한 질문 처리.
- **supervisor()**: 라우터 선택 → 분류 → 위임 → 답변, 라우팅 로그 출력.
- 실무는 **하이브리드**(규칙 먼저, 안 잡히면 LLM)로 비용·정확도 동시 확보.

[신규] 규칙 라우터 vs LLM 라우터 비교

라우팅은 **정답이 있는 분류 문제**입니다. 정답 라벨을 붙인 테스트셋으로 두 라우터의 **정확도**와 **LLM 호출 횟수(비용)**를 나란히 측정합니다. 규칙 라우터는 호출 0회(빠름·공짜)지만 애매한 질문을 놓치고, LLM 라우터는 질문당 1회(비용)지만 정확하죠. 결론은 **"규칙 먼저, 안 잡히면 LLM"** 2단계 전략입니다.

1. 왜 측정하나

05번에서 두 라우터를 만들었지만, "어느 게 얼마나 정확한지" 는 감으로만 알았습니다. 운영 결정을 하려면 숫자가 필요하죠.

- 규칙 라우터는 정말 빠르고 공짜인가? → LLM 호출 0회 확인
- 규칙 라우터는 얼마나 놓치나? → 정확도 측정
- LLM 라우터는 얼마나 정확한가? → 정확도 측정
- 비용은 얼마나 차이어나나? → 호출 횟수 비교

라우팅은 **분류 문제**라, 정답 라벨을 붙이면 **정확도를 객관적으로 잴 수** 있습니다.

2. 정답 라벨 테스트셋

policy/sales를 섞고, 특히 키워드로 안 잡히는 애매한 질문을 일부러 넣어 규칙 라우터의 한계를 드러냅니다.

```
# 실행: python "sections/25_multi_agent_concept/06_규칙라우터-vs-LLM라우터-비교.py"
import sys, pathlib
sys.path.append(str(pathlib.Path(__file__).resolve().parents[2] / "code"))
from common import get_chat, DATA
from ch25_multi_agent import route_rule, route_llm, build_agents # 두 라우터 재사용

# [핵심] 라우팅은 '정답이 있는 분류 문제' → 정답 라벨을 붙이면 정확도를 잴 수 있다.
TESTSET = [
    ("환불은 며칠 안에 신청해야 해?", "policy"), # 키워드 '환불' → 규칙도 맞춤
    ("무료배송 기준이 어떻게 돼?", "policy"), # '무료배송'
    ("포인트 적립은 언제 돼?", "policy"), # '적립/포인트'
    ("교환하려면 어떻게 해?", "policy"), # '교환'
    ("전자기기 추천 좀 해줘", "sales"), # 명확한 추천
    ("패션의류 인기상품 뭐 있어?", "sales"), # 명확한 추천
    ("선물용으로 좋은 거 골라줘", "sales"), # 키워드 없음 → 규칙은 sales(기본)로 맞춤
    ("가성비 좋은 거 없을까?", "sales"), # 애매 → 규칙은 sales(기본)로 우연히 맞춤
    ("주문한 거 무를 수 있어?", "policy"), # '취소' 동의어 → 규칙이 놓치기 쉬움
    ("이거 며칠이면 받아볼 수 있어?", "policy"), # 배송 의미지만 '배송' 단어 없음 → 규칙 한계
]
```

마지막 두 질문이 핵심 함정입니다.

"주문한 거 무를 수 있어?" → 의미는 '취소'(policy)지만 '취소' 단어가 없음
"이거 며칠이면 받아?" → 의미는 '배송'(policy)지만 '배송' 단어가 없음
→ 규칙 라우터는 키워드가 없어 sales로 잘못 보냄 (오분류!)
→ LLM 라우터는 의미를 이해해 policy로 정확히 보냄

3. 평가 함수

두 라우터를 같은 질문셋에 돌려 정확도와 호출 횟수를 집계합니다.

```
def evaluate():
    """두 라우터를 같은 질문셋에 돌려 정확도와 LLM 호출 횟수를 집계한다."""
    llm, _, _ = build_agents() # route_llm 에 넘길 분류용 LLM

    rule_correct = llm_correct = 0
    rule_calls = 0 # 규칙 라우터의 LLM 호출 횟수(항상 0이어야 함)
    llm_calls = 0 # LLM 라우터의 호출 횟수(질문당 1회)

    print(f" {'질문':<26} | 정답 | 규칙 | LLM")
    print(" " + "-" * 56)
    for q, gold in TESTSET:
        r = route_rule(q) # LLM 호출 없음 → rule_calls 증가 안 함
        l = route_llm(llm, q) # 질문당 LLM 1회
        llm_calls += 1

        rule_ok, llm_ok = (r == gold), (l == gold)
        rule_correct += rule_ok
        llm_correct += llm_ok

        mark = lambda ok: "O" if ok else "X"
        qshort = (q[:24] + "...") if len(q) > 25 else q
        print(f" {qshort:<26} | {gold:<6}| {r:<5}{mark(rule_ok)} | {l:<5}{mark(llm_ok)}")

    n = len(TESTSET)
    print(f"\n 규칙 라우터 : 정확도 {rule_correct}/{n} ({rule_correct/n*100:.0f}%), "
          f"LLM 호출 {rule_calls}회 (빠름·공짜)")
    print(f" LLM 라우터 : 정확도 {llm_correct}/{n} ({llm_correct/n*100:.0f}%), "
          f"LLM 호출 {llm_calls}회 (정확하지만 비용)")

if __name__ == "__main__":
    evaluate()
```

핵심 측정 포인트

- rule_calls는 **항상 0** — 규칙 라우터는 LLM을 안 부릅니다(키워드 매칭만).
- llm_calls는 **질문 수만큼**(여기선 10) — 매 질문마다 LLM 1회 호출.
- 정확도는 예측 == 정답 카운트로 계산.

4. 실행 결과

예상 출력

질문	정답	규칙	LLM	
환불은 며칠 안에 신청해야 해?	policy	policy0	policy0	
무료배송 기준이 어떻게 돼?	policy	policy0	policy0	
포인트 적립은 언제 돼?	policy	policy0	policy0	
교환하려면 어떻게 해?	policy	policy0	policy0	
전자기기 추천 좀 해줘	sales	sales	0	sales 0
패션의류 인기상품 뭐 있어?	sales	sales	0	sales 0
선물용으로 좋은 거 골라줘	sales	sales	0	sales 0
가성비 좋은 거 없을까?	sales	sales	0	sales 0
주문한 거 무를 수 있어?	policy	sales X	policy0	← 규칙 놓침!
이거 며칠이면 받아볼 수 있어?	policy	sales X	policy0	← 규칙 놓침!
규칙 라우터 : 정확도 8/10 (80%), LLM 호출 0회 (빠름 · 공짜)				
LLM 라우터 : 정확도 10/10 (100%), LLM 호출 10회 (정확하지만 비용)				

관찰 포인트

- 규칙 라우터: 80% 정확, LLM 호출 0회 → 명확한 질문은 완벽, 애매한 2개를 놓침
- LLM 라우터: 100% 정확, LLM 호출 10회 → 다 맞췄지만 매번 비용

마지막 두 질문("무를 수 있어?", "며칠이면 받아?")에서 규칙 라우터가 키워드가 없어 놓쳤습니다. LLM 라우터는 의미를 이해해 맞췄죠.

5. 결론 — 2단계 하이브리드

이 측정이 알려주는 운영 전략은 명확합니다.

규칙 라우터: 빠름 · 공짜 but 애매한 질문 놓침

LLM 라우터: 정확 but 매 질문 비용

↓ 둘의 장점만

하이브리드: 명확한 질문은 규칙(공짜)으로, 애매한 질문만 LLM으로

```
def route_hybrid(llm, question: str) -> str:
    if any(w in question for w in POLICY_WORDS): # 명확한 키워드 → 공짜로 즉시
        return "policy"
    return route_llm(llm, question) # 애매한 것만 LLM(비용)
```

대부분의 질문(80%)은 규칙으로 공짜 처리하고, 나머지 애매한 20%만 LLM에 물으면:

- 정확도: ~100% (LLM이 애매한 것 처리)
- 비용: ~20% (질문 10개 중 2개만 LLM 호출)

정확도와 비용을 둘 다 잡습니다. 이게 측정 기반 의사결정의 힘이지요 — 감이 아니라 숫자로 전략을 정합니다.

6. 측정의 일반 교훈

이 비교가 주는 더 큰 교훈:

- "정확한가?"는 측정해야 한다 (감 ❌, 정답 라벨 + 정확도 ✅)
- "비용은?"도 측정해야 한다 (LLM 호출 횟수 카운트)
- 트레이드오프(정확도 vs 비용)는 숫자로 봐야 결정할 수 있다

라우터뿐 아니라 모든 AI 컴포넌트(분류·검색·추천)에 적용됩니다. "이 방식이 더 나을 것 같다"가 아니라 "측정해보니 N% 정확하고 M원 든다"로 말하는 게 엔지니어링이지요.